

CHAPTER 13

Verilog 2001 增強特色

Verilog

本章介紹的是 Verilog 2001 增強特色，還有 Verilog 2001 新增的保留字、運算子、函數、compiler directive 以及 token。

Verilog 1.0 就是 IEEE Std. 1364-1995 Verilog standard，也就一般所稱的 Verilog 1995。

Verilog 2.0 則是 IEEE Std. 1364-2001 Verilog standard，也就一般所稱的 Verilog 2001，它是 Verilog 1995 硬體描述語言的增強版，一般稱它為 Verilog 2001。

Verilog 2001 是以 Verilog 1995 為基礎，在不影響原先的電路設計以及電路的模擬速度之下，讓 Verilog 硬體描述語言功能更強、更容易使用。

Verilog 2001 的目標：

■ 功能增強

更高階，抽象性系統層次的 modeling

Intellectual Property (IP) modeling

對於非常深次微米 (very deep submicron) 的 timing 正確性

■ 讓 Verilog 更容易使用

■ 更正既有之錯誤以及更正模稜兩可的 Verilog 敘述／寫法

■ 與既有的 Verilog 設計相容

■ 保證電路模擬以及合成軟體的廠商 (EDA vender) 設計出來的軟體都會支援 Verilog-2001

『參考資料來源』

① http://www.model.com/products/verilog_ad/verilog_2001.asp

② http://www.simucad.com/verilog_press_releases/case_study

shtml

- ③ www.verilog-2001.com/verilog-2001_presentation.pdf
- ④ www.sutherland-hdl.com/papers/2001-Wescon-tutorial_using_Verilog-2001_part1.pdf
- ⑤ www.lcdm-eng.com/SJ_2001.pdf
- ⑥ www.sutherland-hdl.com/papers/2000-HDLCon-paper_Verilog-2000.pdf
- ⑦ www.sutherland-hdl.com/papers/2001-SNUG-presentation_Verilog-2000_standard.pdf

Verilog 2001 共有 33 項增強的部份，分別描述在下列各個小節中。



13.1 Configuration

Verilog 1995 以及之前並沒有像 VHDL 有 configuration 這一類的敘述。

configuration 通常會在 test bench 測試程式中看得到，它主要的功能用於選擇已經事先設計好的 module，而這些功能相同的 module 通常會有一種以上的設計方式，為了驗證不同的設計方式電路模擬的結果是否相同就會用到 config 敘述。

比方說某個您可能先用 Behavioral Level 的方式來設計一個 design，也可以 RTL Level 的方式來設計，也可能用 Structural Level 的方式去設計；不同的設計方式電路模擬的結果也要相同才能保證您用三種不同 Level 設計出來的 design 是相同功能的，還有另外一種可能是您使用不能合成的 VHDL

語法寫出一個功能模擬 (functional simulation) 的寫法，只是爲了驗證您的電路動作情形是否相同；所以會用到 `config` 敘述選擇一個設計中的不同的 `module` 來作電路的模擬，兩兩之間電路模擬的結果就可拿來作比較與驗證了。

`Configuration` 可以用來指定某個 `module` 可以引用那個 `instance`。

`Configuration` 敘述必須寫在所有的 `module` 之外，可以和 Verilog 的 `module` 寫在同一個檔案中，也可以分別在不同的檔案裡頭。

新增加的 Verilog 保留字：`config`、`endconfig`、`design`、`instance`、`cell`、`use` 以及 `liblist`。

可惜的是 ModelSim SE 5.6b 這個版本並沒有支援 Verilog 2001 的 `Configuration` 這一系列的敘述，用 ModelSim SE 5.7 版也試不成功。

將來 Verilog 2001 的“`Configuration`”是可以作爲電路合成的。

Configuration 的例子

【Design】

```
module myChip (...);  
    ...  
    adder a1 (...);  
    adder a2 (...);  
    ...  
endmodule  
  
module test;  
    ...  
    myChip dut (...);  
    ...  
endmodule
```


【configuration 定義(它可以是 Source Code 的一部份，也就是可以寫在 .v 中)】

```
/* 定義 configuration 的名稱 */  
config cfg4  
  
/* 指定最上層 module 的名稱 */  
design rtlLib.test  
  
/* 設定尋找 module 的順序 */  
/* 以底下的例子：找尋某個 module 先找 liblist、再找 rtlLib，最後才找 gateLib */  
default liblist rtlLib gateLib;  
  
/* 外顯示地指定 module instance 所使用的 library */  
instance test.dut.a2 liblist gateLib;  
  
/* 結束 configuration 區塊 */  
endconfig
```

【Library Map (對映) 的檔案內容(它不是 Source Code 的一部份，也就是不能寫在 .v 中)】

```
/* 指定 RTL model 檔案的所在位置與檔案名稱 */  
/* 以底下的例子：是在目前目錄下的 .v 檔 */  
library rtlLib ./*.v;  
  
/* 指定 電路合成之後 Gate Level model 檔案的所在位置與檔案名稱 */  
/* 以底下的例子：是在目前目錄下的 synth_out 目錄下的 .v 檔 */  
library gateLib ./synth_out/*.v;
```



13.2 generate

generate 敘述通常會搭配 for 迴圈敘述用來產生一連串的：module、primitive、procedure、task、function、variable、wire 以及訊號指定敘述。

generate 主要用來作“複製”的動作，也可以搭配 if ... else 以及 case 敘述來設計您的電路，使得整個 Verilog 2001 的 Generate 敘述比 VHDL 來得強些。

新增加的 Verilog 保留字：generate、endgenerate、genvar 以及 localparam。

Verilog 2001 的“generate”是可以作為電路合成的。

Quartus II 2.1 版，有支援 generate、genvar 以及 localparam。

Synplify 7.1 版以及 Synplify Pro 7.1 版，有支援 generate、genvar 以及 localparam。

SIMUCAD 的 Silos Verilog Logic Simulator，有支援 generate、endgenerate、genvar 以及 localparam。

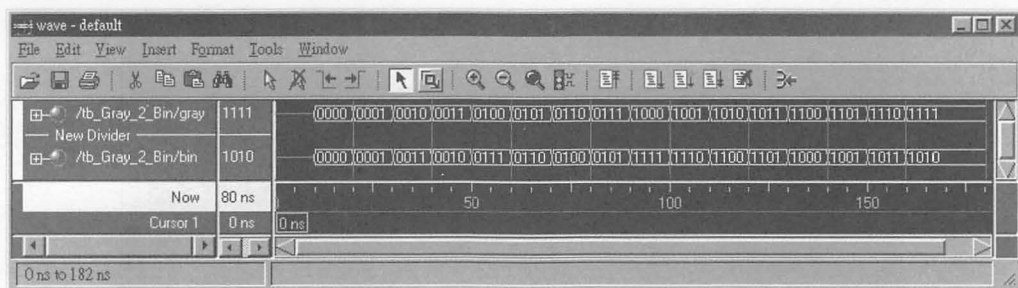
ModelSim SE 5.7 之後的版本才有支援 generate、endgenerate 以及 genvar。

而 ModelSim SE 5.5 之後的版本就有支援 localparam 了。

generate 的例子 1

```
module Gray_2_Bin (gray, bin);
    parameter SIZE = 4;
    input  [SIZE-1:0] gray;
    output [SIZE-1:0] bin;

    genvar i;
    generate
        for( i=0; i<SIZE; i=i+1 )
            begin: bit
                assign bin[i] = ^gray[SIZE-1-i];
            end
    endgenerate
endmodule
```



generate 的例子 2

```

module Nbit_adder (a, b, ci, sum, co);
    parameter SIZE = 4;
    input  [SIZE-1:0] a, b;
    input      ci;
    output [SIZE-1:0] sum;
    output      co;

    wire [SIZE:0] c;

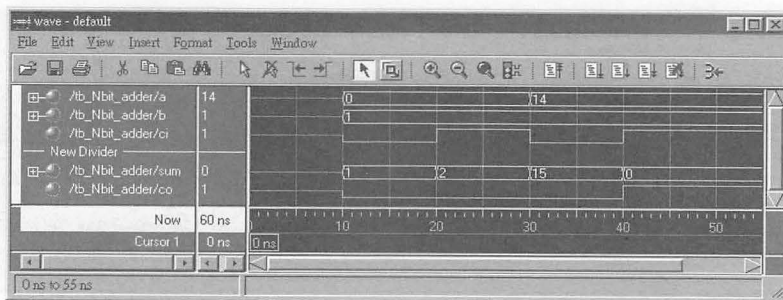
    genvar i;
    assign c[0] = ci;

    assign co = c[SIZE];

    generate
        for(i=0; i<SIZE; i=i+1)
            begin: addbit
                wire n1, n2, n3; // 内部的連線
                xor g1(n1, a[i], b[i]);
                xor g2(sum[i], n1, c[i]);
                and g3(n2, a[i], b[i]);
                and g4(n3, n1, c[i]);
                or  g5(c[i+1], n2, n3);
            end
    endgenerate

endmodule

```



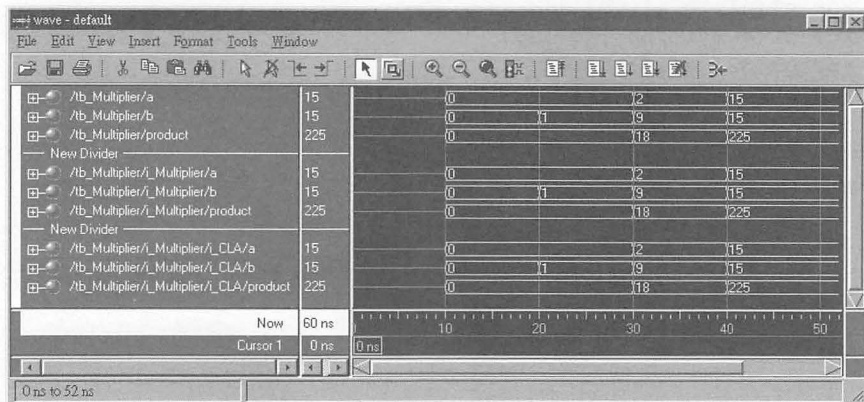
generate 的例子 3

```

module Multiplier (a, b, product);
    parameter a_width = 8, b_width = 8;
    /* localparam 是常數、不能改變，parameter 可由引用時改變 */
    localparam product_width = a_width + b_width;
    input [a_width-1:0] a;
    input [b_width-1:0] b;
    output [product_width-1:0] product;

    generate
        if ((a_width < 8) || (b_width < 8))
            /* 只要 a 或 b 的寬度小於 8 則用 Carry-Look-Ahead 乘法器 */
            CLA_multiplier #(a_width, b_width) u1 (a, b, product);
        else
            /* 否則用 WALLACE-tree 乘法器 */
            WALLACE_multiplier #(a_width, b_width) u1 (a, b, product);
    endgenerate
endmodule

```





13.3 Constant function

constant function 的語法和標準 Verilog 的 function 語法相同，constant function 的運算結果是在“編譯”的時候就決定了，它可以用在：向量寬度宣告 (Vector width declaration)、陣列大小宣告 (Array declaration) 以及重覆運作 (Replicate operation)，讓您設計出更有彈性 (scalable)、可重覆使用 (reusable) 的 module。

將來 Verilog 2001 的 “constant function” 是可作為電路合成的。

可惜的是 ModelSim SE 5.7c 這個版本並沒有支援“使用較複雜的 constant function”這種寫法。

constant function 的例子

【Verilog 1995 使用簡單的 constant 運算式】

```
module ram_1 (CS, RW, Address, Data_In, Data_Out);
  parameter ADDRESS_SIZE = 8;
  parameter WIDTH = 8;
  parameter SIZE = 256;

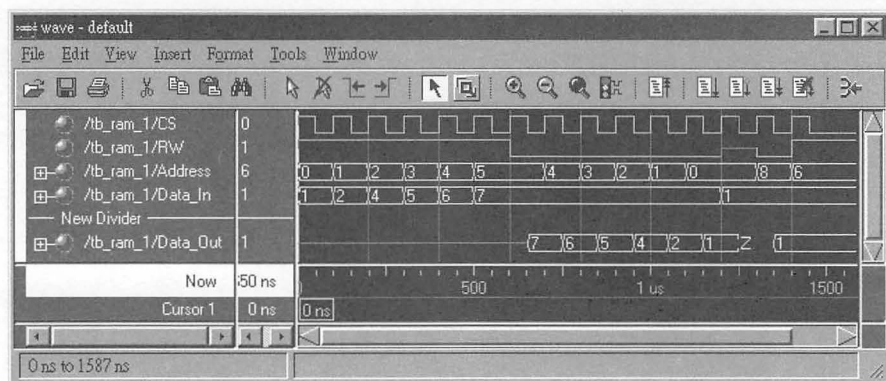
  input CS, RW;
  /* Verilog 1995 使用簡單的 constant 運算式 */
  input [ADDRESS_SIZE-1:0] Address;
  input [WIDTH-1:0] Data_In;
  output [WIDTH-1:0] Data_Out;
  reg [WIDTH-1:0] Data_Out;
  reg [WIDTH-1:0] RAM_Data[SIZE-1:0];

  always @(negedge CS)
  begin // 負緣觸發，選擇到此晶片
    if (RW == 1'b0) // 讀取
    begin
      Data_Out = RAM_Data[Address[2:0]];
    end
    else
    begin
```

```

    if (RW == 1'b1) // 寫入
    begin
        RAM_Data[Address[2:0]] = Data_In;
    end
    else
        Data_Out = 3'bz; // 高阻抗 (High Impedance)
    end
end
endmodule

```



【Verilog 2001 使用較複雜的 constant function 例子 1】

// ModelSim SE 5.7c 沒有支援 "使用較複雜的 constant function"

```

module ram_2 (CS, RW, Address, Data_In, Data_Out);
    parameter WIDTH = 8;
    parameter SIZE = 256;

```

```

    input CS, RW;

```

// ModelSim 5.7c 沒有支援 "使用較複雜的 constant function"

```

// parameter RAM_SIZE = 1024;
// input [clogb2(RAM_SIZE)-1:0] Address;

```

// 必須改成底下的寫法:

```

parameter ADDRESS_SIZE = 8;
input [ADDRESS_SIZE-1:0] Address;

```

.....

```

function integer clogb2;
  input depth;
  integer i;
  begin
    clogb2 = 1;
    for (i = 0; 2**i < depth; i = i + 1)
      clogb2 = i + 1;
  end
endfunction

.....

endmodule

```

【Verilog 2001 使用較複雜的 constant function 例子 2】

```

// ModelSim SE 5.7c 沒有支援 "使用較複雜的 constant function"
module ram_3 (CS, RW, Address, Data_In, Data_Out);
  parameter WIDTH = 8;
  parameter SIZE = 256;

  // ModelSim 5.7c 沒有支援 "使用較複雜的 constant function"
  // localparam ADDRESS_SIZE = clogb2(SIZE);

  // 必須改成底下的寫法:
  parameter ADDRESS_SIZE = 8;

  .....

  input [ADDRESS_SIZE-1:0] Address;
  .....

  function integer clogb2;
    input depth;
    integer i;
    begin
      clogb2 = 1;
      for (i = 0; 2**i < depth; i = i + 1)
        clogb2 = i + 1;
    end
  endfunction

  .....

endmodule

```




13.4 Indexed vector part select

Verilog 2001 可以用 register 的值當作變數來去選擇向量 (vector) 中的某些範圍，其中：範圍的“起始點”可以視 register 的值做動態改變的，而範圍的大小則必須是常數。

將來 Verilog 2001 的“Indexed vector part select”是可以作為電路合成的。

Quartus II 2.1 版，有支援“Indexed vector part select”。

ModelSim SE 5.6 之後的版本才有支援“Indexed vector part select”。

Indexed vector part select 的說明

```
reg [7:0] Data1;
reg [0:7] Data2;
```

因為 [7:0] Data1, 所以 Data1[2 +: 2] 等於 Data1[3:2]

因為 [7:0] Data1, 所以 Data1[2 -: 2] 等於 Data1[2:1]

因為 [0:7] Data2, 所以 Data2[2 +: 2] 等於 Data2[2:3]

因為 [0:7] Data2, 所以 Data2[2 -: 2] 等於 Data2[1:2]

其中：+：表示從起始點“增加” part-select

-：表示從起始點“減少” part-select

Indexed vector part select 的例子

```
module part_select (bit_address, data_in, byte_num, byte_add_1,
byte_sub_1, byte_add_2, byte_sub_2);
  input [5:0] bit_address; // bit_address 是 0 ~ 63
  input      data_in;
  input [2:0] byte_num;    // byte_num 是 0 ~ 7
  output [7:0] byte_add_1;
  output [7:0] byte_sub_1;
```

```

output [7:0] byte_add_2;
output [7:0] byte_sub_2;

reg [63:0] word_1;
reg [0:63] word_2;

always @(bit_address or data_in)
begin
    word_1[bit_address] = data_in;
    word_2[bit_address] = data_in;
end

wire [7:0] byte_add_1 = word_1[byte_num*8 +: 8];
// 因為 [63:0] word_1
// 當 byte_num 等於 0 則 [7:0] byte_add_1 = word_1[7:0]

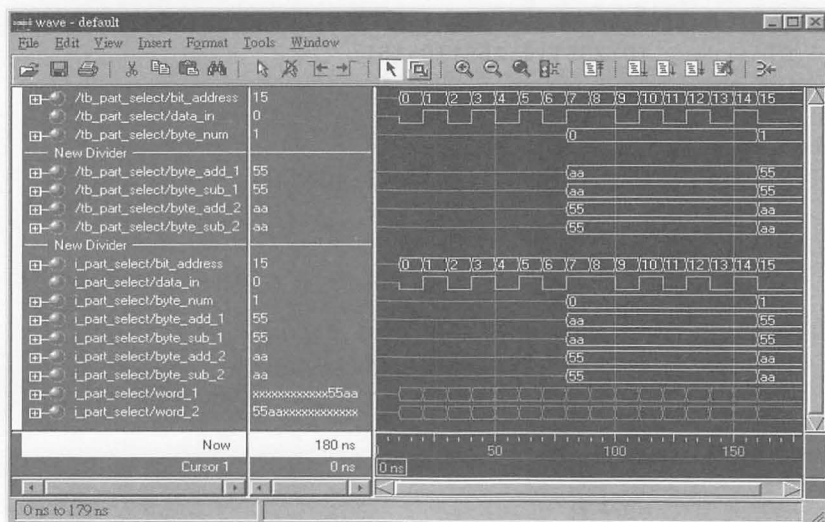
wire [7:0] byte_sub_1 = word_1[(byte_num+1)*8-1 -: 8];
// 因為 [63:0] word_1
// 當 byte_num 等於 0 則 [7:0] byte_sub_1 也是 = word_1[7:0]

wire [7:0] byte_add_2 = word_2[byte_num*8 +: 8];
// 因為 [0:63] word_2
// 當 byte_num 等於 0 則 [7:0] byte_add_2 = word_2[0:7]

wire [7:0] byte_sub_2 = word_2[(byte_num+1)*8-1 -: 8];
// 因為 [0:63] word_2
// 當 byte_num 等於 0 則 [7:0] byte_sub_2 也是 = word_2[0:7]

endmodule

```





13.5 Multi-dimensional Array

Verilog 1995 只提供“一維陣列”的設計方式讓您去產生出：一維的 reg、wire、integer 或者是 time... 等等資料，通常用它來 model 隨機存取記憶體 (RAM) 以及唯讀記憶體 (ROM)。

Verilog 2001 除了可以使用“一維陣列”以外，也可以讓您使用“多維陣列”的設計方式，來產生各種資料型態的“多維陣列”。

Verilog 2001 的“Multi-dimensional Array”可以作為電路合成。

ModelSim SE 5.6 之後的版本才有支援“Multi-dimensional Array”，而且 Array 也可以是 wire 及 real 這二種資料型態，例如：

```
wire [63:0] w_array [15:0];  
real r_array [1:10][1:4][1:4];
```

Multi-dimensional Array 的說明

```
// 宣告一個具有 8 個位元 wire 的三維陣列 (0~255) (0~255) (0~15)  
wire [7:0] array1 [0:255][0:255][0:15];  
  
// 選用三維陣列中的其中一個 8 個位元 wire  
parameter addr1=1, addr2 = 2, addr3 =3;  
wire [7:0] out1 = array1[addr1][addr2][addr3];  
  
// 宣告一個具有 32 個位元 reg 的二維陣列 (0~255) (0~15)  
reg [31:0] array2 [0:255][0:15];  
  
// 選用二維陣列中的其中一個 32 個位元中的最高 8 個位元形成 wire [7:0]  
wire [7:0] out2 = array2[100][7][31:24];
```

Multi-dimensional Array 的例子

```

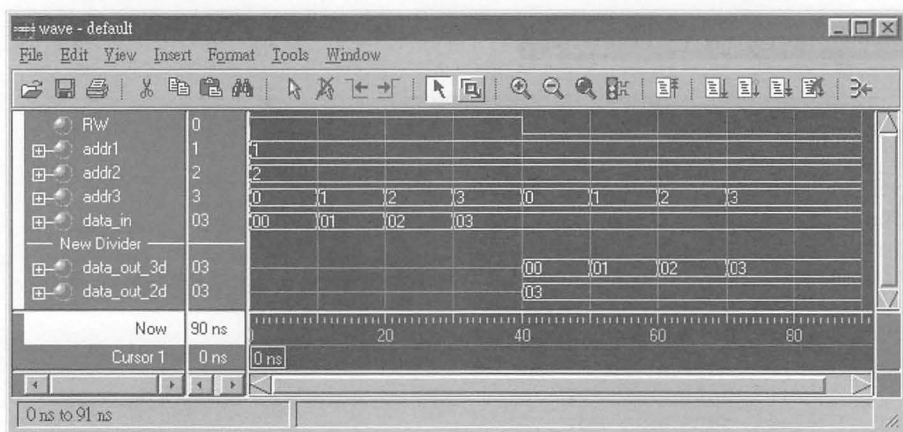
module MD_Array (RW, addr1, addr2, addr3, data_in, data_out_3d,
data_out_2d);
    input      RW;      // Read 或 Write
    input [2:0] addr1;  // addr1 是 0 ~ 7
    input [2:0] addr2;  // addr2 是 0 ~ 7
    input [2:0] addr3;  // addr3 是 0 ~ 7
    input [7:0] data_in;
    output [7:0] data_out_3d;
    output [7:0] data_out_2d;
    reg [7:0] data_out_3d;
    reg [7:0] data_out_2d;

    // 宣告一個具有 8 個位元 reg 的三維陣列 (0~7) (0~7) (0~7)
    reg [7:0] array_3d [0:7][0:7][0:7];

    // 宣告一個具有 32 個位元 reg 的二維陣列 (0~7) (0~7)
    reg [7:0] array_2d [0:7][0:7];

    always @(RW or addr1 or addr2 or addr3 or data_in)
    begin
        if (RW == 1'b1) // 寫入
        begin
            array_3d[addr1][addr2][addr3] = data_in;
            array_2d[addr1][addr2] = data_in;
        end
        else
        begin
            if (RW == 1'b0) // 讀取
            begin
                data_out_3d = array_3d[addr1][addr2][addr3];
                data_out_2d = array_2d[addr1][addr2];
            end
            else
            begin
                data_out_3d = 8'bz; // 高阻抗 (High Impedance)
                data_out_2d = 8'bz; // 高阻抗 (High Impedance)
            end
        end
    end
end
endmodule

```



13.6 Array Bit and Part Select

Verilog 2001 增加了對於“多維陣列”的位元選擇(Bit-select)與部份選擇(Part-select)的運作。

Verilog 2001 的“Array Bit and Part Select”是可以作為電路合成的。

ModelSim SE 5.6 之後的版本就有支援“Array Bit and Part Select”。

Array Bit and Part Select 的說明

```
// 宣告一個具有 32 個位元 reg 的二維陣列 (0~255) (0~15)
reg [31:0] array2 [0:255][0:15];

// 取得 32 個位元 reg 二維陣列中索引位址 (100) (7) 的最低 1 個位元 reg
wire out1 = array2[100][7][0]; // Bit Select

// 取得 32 個位元 reg 二維陣列中索引位址 (100) (7) 的最高 8 個位元 reg
wire [7:0] out2 = array2[100][7][31:24]; // Part Select
```

Array Bit and Part Select 的例子

```

module Array_Bit_Part (RW, addr1, addr2, addr3, data_in,
data_out_3d, data_
out_2d, bit0_3d, bit3_0_3d);
    input      RW; // Read 或 Write
    input  [2:0] addr1; // addr1 是 0 ~ 7
    input  [2:0] addr2; // addr2 是 0 ~ 7
    input  [2:0] addr3; // addr3 是 0 ~ 7
    input  [7:0] data_in;
    output [7:0] data_out_3d;
    output [7:0] data_out_2d;
    output      bit0_3d;
    output [3:0] bit3_0_3d;
    reg  [7:0] data_out_3d;
    reg  [7:0] data_out_2d;
    reg      bit0_3d;
    reg  [3:0] bit3_0_3d;

    // 宣告一個具有 8 個位元 reg 的三維陣列 (0~7) (0~7) (0~7)
    reg [7:0] array_3d [0:7][0:7][0:7];

    // 宣告一個具有 8 個位元 reg 的二維陣列 (0~7) (0~7)
    reg [7:0] array_2d [0:7][0:7];

    always @(RW or addr1 or addr2 or addr3 or data_in)
    begin
        if (RW == 1'b1) // 寫入
        begin
            array_3d[addr1][addr2][addr3] = data_in;
            array_2d[addr1][addr2] = data_in;
        end
        else
        begin
            if (RW == 1'b0) // 讀取
            begin
                data_out_3d = array_3d[addr1][addr2][addr3];
                data_out_2d = array_2d[addr1][addr2];
                bit0_3d = array_3d[addr1][addr2][addr3][0];
                bit3_0_3d = array_3d[addr1][addr2][addr3][3:0];
            end
            else
            begin
                data_out_3d = 8'bz; // 高阻抗 (High Impedance)
            end
        end
    end

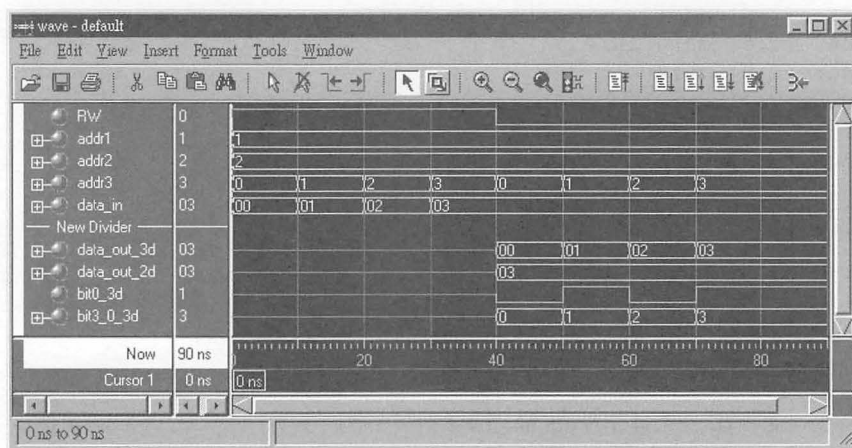
```

```

        data_out_2d = 8'bz; // 高阻抗 (High Impedance)
        bit0_3d    = 1'bz;
        bit3_0_3d  = 4'bz;
    end
end
end

endmodule

```



13.7 Signed Arithmetic Extension

Verilog 2001 增加了 reg 及 wire 這二種資料型態可以宣告成 signed (有號數)。

Verilog 2001 的 “Signed Arithmetic Extension” 是否能夠作為電路合成，得看 EDA tool 廠商要不要設計而定。

Quartus II 2.1 版，有支援 “Signed Arithmetic Extension”。

Quartus II 2.1 版，也有支援 “有號數” 的算術位移運算元 <<< 和 >>>。

Synplify 7.1 版及 Synplify Pro 7.1 版，有支援 “Signed Arithmetic Extension”。

ModelSim SE 5.5 之後的版本就有支援 “Signed Arithmetic Extension”，當然也包括 \$signed() 轉換成 signed (有號數) 及 \$unsigned() 轉換成 unsigned (無號數)，還有支援 “Signed based integer number”，例如：`data = -4'sHf;` // 等於 $-(4'b1111) = -(-1) = 1$ 。

Signed Arithmetic Extension 的說明

(1) reg 以及 wire

```
reg signed [15:0] data;  
wire signed [9:0] address;
```

(2) 整數值用 signed 表示

```
reg [15:0] data;  
data = 16'sHC501; // s: signed (有號數), H: Hexadecimal (16  
進制值)
```

(3) function 的傳回值也可以是 signed

```
function signed [128:0] signed_func;
```

(4) module 的輸入埠也可以是 signed

```
module signed_module( data );  
  input signed [7:0] data  
  ...  
endmodule  
  
signed_module u1(8'sH08); // 叫用 signed_module
```

(5) “有號數”的算術位移運算元

新增加的算術移位運算子：`>>>` 和 `<<<`。

>>> 是算術右移運算子，它會將高位元的部份補上原先在最高位元的值(MSB)，也就是用符號位元在填補。不同於 Verilog 1995 的 >> 右移運算子，它會將高位元的部份補上 '0'。

<<< 是算術左移運算子，它會將低位元的部份補上 '0'。和 Verilog 1995 的 << 左移運算子功能相同。

例如：

```
in = 8'b11001010;
always @(in)
begin
    out1 = in >> 3;    // out1 會等於 'b00011001
    out2 = in >>> 3;   // out2 會等於 'b11111001
    out3 = in << 3;    // out3 會等於 'b01010000
    out4 = in <<< 3;   // out4 會等於 'b01010000
end
```

新增加的資料型態轉換函數：\$signed() 和 \$unsigned()。

\$signed() 轉換成 signed(有號數)

\$unsigned() 轉換成 unsigned(無號數)

例如：

```
$signed(Data1 - Data2)
$unsigned(8'sHff); // 等於 $unsigned(-255) = 255
```

Signed Arithmetic Extension 的例子 1

```
module Signed_Test (RW, addr1, addr2, data_in, data_in2,
                    signed_data1, signed_data2, in,
                    data_out_2d, data_out_1d, data_out_signed,
                    out1, out2, out3, out4, add_2_signed,
                    add_2_unsigned);
    input          RW;          // Read 或 Write
    input signed [2:0] addr1;    // addr1 是 -4 ~ 3
    input [2:0]    addr2;       // addr2 是 0 ~ 7
    input [7:0]    data_in;
```

```

input          [7:0] data_in2;
input signed [7:0] signed_data1; // -128 ~ 127
input signed [7:0] signed_data2; // -128 ~ 127
input signed [7:0] in;
output         [7:0] data_out_2d;
output         [7:0] data_out_1d;
output signed [8:0] data_out_signed;
output         [7:0] out1, out2, out3, out4;
output signed [8:0] add_2_signed;
output         [8:0] add_2_unsigned;
reg            [7:0] data_out_2d;
reg            [7:0] data_out_1d;
reg            [8:0] data_out_signed;
reg signed [7:0] out1, out2, out3, out4;
reg signed [8:0] add_2_signed;
reg            [8:0] add_2_unsigned;

// for test only, no use
wire signed [ 9:0] address;

// 宣告一個具有 8 個位元 reg 的二維陣列 (-4~3) (0~7)
reg [7:0] array_2d [-4:3][0:7];

// 宣告一個具有 8 個位元 reg 的二維陣列 (-4~3)
reg [7:0] array_1d [-4:3];

always @(RW or addr1 or addr2 or data_in)
begin
    if (RW == 1'b1) // 寫入
    begin
        array_2d[addr1][addr2] = data_in;
        array_1d[addr1] = data_in;
    end
    else
    begin
        if (RW == 1'b0) // 讀取
        begin
            data_out_2d = array_2d[addr1][addr2];
            data_out_1d = array_1d[addr1];
        end
        else
        begin
            data_out_2d = 8'bz; // 高阻抗 (High Impedance)
            data_out_1d = 8'bz; // 高阻抗 (High Impedance)
        end
    end
end

function signed [8:0] signed_add; // -256 ~ 255
input signed [7:0] data1; // -128 ~ 127

```

```

input signed [7:0] data2;          // -128 ~ 127
begin
    signed_add = data1 + data2;
end
endfunction

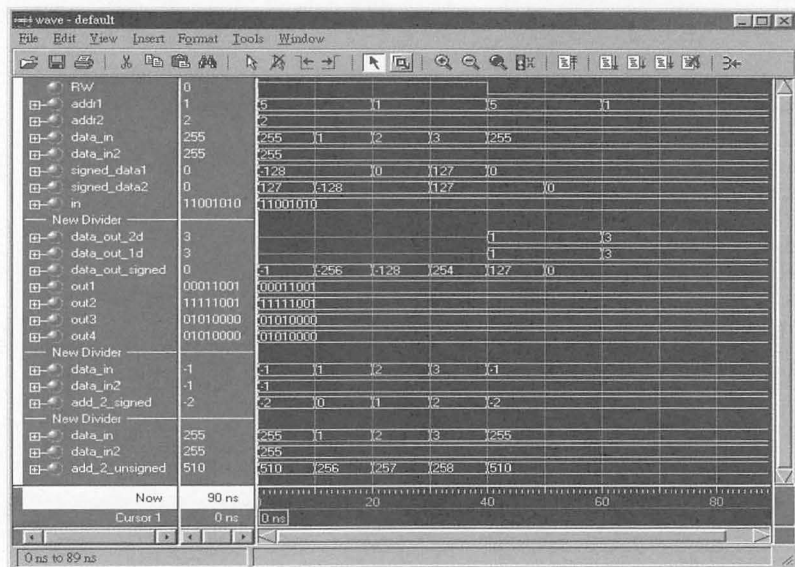
always @(signed_data1 or signed_data2)
begin
    data_out_signed = signed_add(signed_data1, signed_data2);
end

always @(in)                      // 如果 in = 8'b11001010
begin
    out1 = in >> 3;               // out1 會等於 'b00011001
    out2 = in >>> 3;              // out2 會等於 'b11111001
    out3 = in << 3;               // out3 會等於 'b01010000
    out4 = in <<< 3;              // out4 會等於 'b01010000
end

always @(data_in or data_in2)
begin
    add_2_signed = $signed(data_in) + $signed(data_in2);
    add_2_unsigned = $unsigned(data_in) + $unsigned(data_in2);
end

endmodule

```



Signed Arithmetic Extension 的例子 2

```

reg [3:0] m, n;
integer i;
real r;

reg [3:0] m_d_n, signed_m_d_i;
integer m_d_i, five_d_i;
real m_d_2, five_d_2;

initial
begin
    m = 4'b0101; // 5
    n = 4'b0010; // 2
    i = -2;
    r = -2.0;

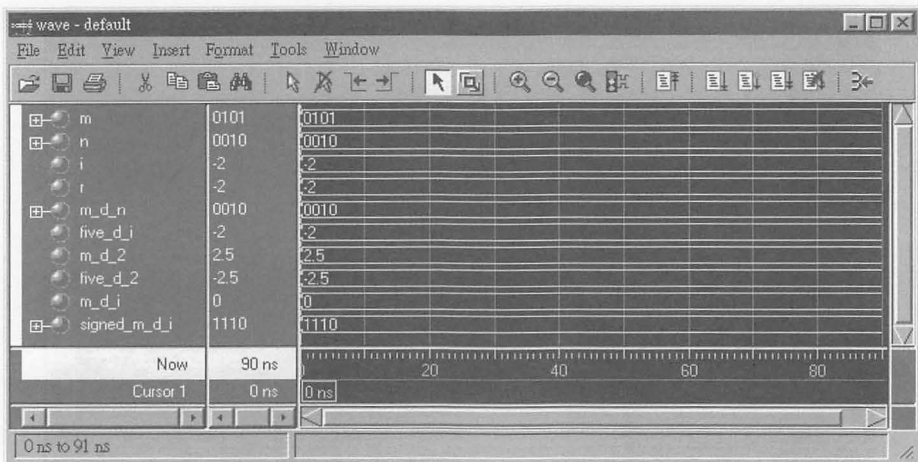
    m_d_n = m / n; // 4'b0010
    five_d_i = 5 / i; // -2
    m_d_2 = m / 2.0; // 2.5
    five_d_2 = 5 / r; // -2.5

    m_d_i = m / i; // 0

    signed_m_d_i = $signed(m) / i; // 4'b1110 = -2

    #20 $finish;
end

```



Signed Arithmetic Extension 的例子 3

【Verilog 1995 的寫法】

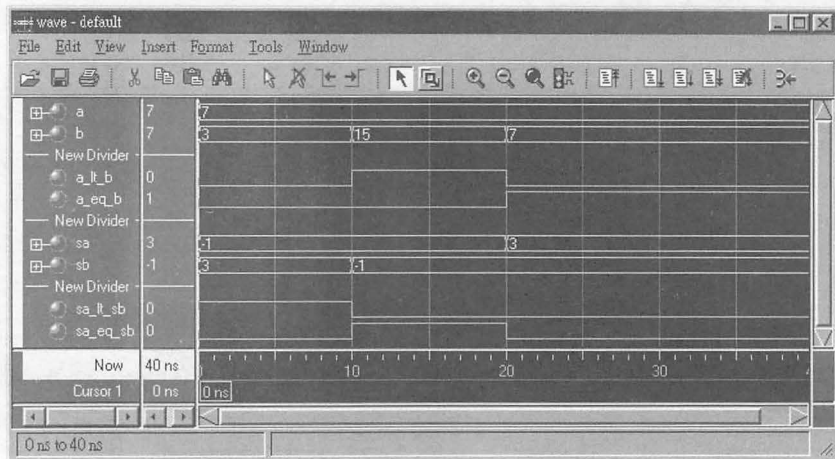
```
module unsigned_mult(a, b, out);
  input  [ 7:0] a;
  input  [ 7:0] b;
  output [15:0] out;

  always @(a or b)
  begin
    out = a * b;
  end
endmodule
```

【Verilog 2001 的寫法】

```
module signed_mult(s_a, s_b, s_out);
  input  signed [ 7:0] s_a;
  input  signed [ 7:0] s_b;
  output signed [15:0] s_out;

  always @(s_a or s_b)
  begin
    s_out = s_a * s_b;
  end
endmodule
```



Signed Arithmetic Extension 的例子 4

```
if (3'sb111 < 3'sb011)      // 判斷條件成立,  -1 <  3 ?
if (3'b111 < 3'b011)      // 判斷條件不成立,  7 <  3 ?
if (3'sb111 == 4'sb1111)   // 判斷條件成立,  -1 == -1 ?
if (3'b111 == 4'b1111)     // 判斷條件不成立,  3 == 15 ?

module Unsigned_Signed_Comp (a, b, a_lt_b, a_eq_b, sa, sb, sa_lt_sb,
sa_eq_sb);
  input      [ 3:0]  a, b;
  output     a_lt_b, a_eq_b;
  reg        a_lt_b, a_eq_b;

  input signed [ 3:0] sa, sb;
  output     sa_lt_sb, sa_eq_sb;
  reg        sa_lt_sb, sa_eq_sb;

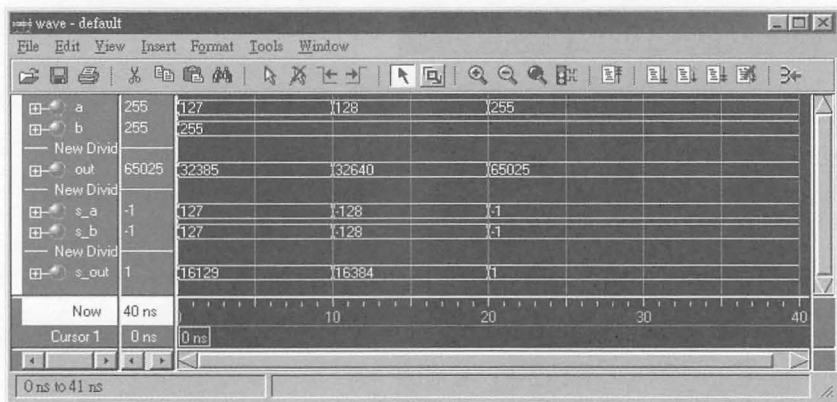
  // if (4'b0111 < 4'b0011)    // 判斷條件不成立,  7 <  3 ?
  always @(a or b)
  begin
    a_lt_b = (a < b);
  end

  // if (4'b0011 == 4'b1111)   // 判斷條件不成立,  3 == 15 ?
  always @(a or b)
  begin
    a_eq_b = (a == b);
  end

  // if (4'sb1111 < 4'sb0011)  // 判斷條件成立,  -1 <  3 ?
  always @(sa or sb)
  begin
    sa_lt_sb = (sa < sb);
  end

  // if (4'sb1111 == 4'sb1111) // 判斷條件成立,  -1 == -1 ?
  always @(sa or sb)
  begin
    sa_eq_sb = (sa == sb);
  end

endmodule
```



13.8 Power Operator

Verilog 2001 增加了“乘冪”(power)運算子 **, 和 C 語言中的 pow() 函數相同。

可以是“整數”的冪次運算, 也可以是“實數”的冪次運算。

Verilog 2001 的“Power Operator”可以作為電路合成。

Quartus II 2.1 版, 有支援“乘冪”(power)運算子 **。

Synplify 7.1 版及 Synplify Pro 7.1 版, 有支援“乘冪”(power)運算子 **。

ModelSim SE 5.5 之後的版本有支援“乘冪”(power)運算子 **。

Power Operator 的例子 1

```
module ram (...);
parameter WORD_SIZE = 64;
parameter ADDR_SIZE = 24;
reg [WORD_SIZE-1:0] core [0:(2**ADDR_SIZE)-1];
// 相當於是 0 ~ 2 的 24 次方 - 1
...
```

Power Operator 的例子 2

```
real i;  
initial i = 2 ** 16;
```

Power Operator 的例子 3

```
module exponent_2_by_Data(Data, Result);  
    input  [3:0] Data;  
    output [7:0] Result;  
    wire  [7:0] Result;  
  
    assign Result = 2 ** Data;  
endmodule
```

Power Operator 的例子 1~例子 3 實作

```
// test bench  
module tb_exponent_2_by_Data;  
  
    reg [ 3:0] Data;    // 0 ~ 15  
    wire [15:0] Result; //  $2^{15} = 32768$ , needing 16 Bits  
  
    exponent_2_by_Data i_exponent_2_by_Data (.Data(Data),  
        .Result(Result));  
    real i;  
    initial i = 2 ** 16;  
  
    parameter WORD_SIZE = 4;  
    parameter ADDR_SIZE = 3;  
    reg [WORD_SIZE-1:0] core [0:(2**ADDR_SIZE)-1];  
    // 相當於是 0 ~ 2 的 3 次方 - 1  
    initial  
    begin  
        core[0] = 4'b0001;  
        core[1] = 4'b0010;  
        core[2] = 4'b0011;  
        core[3] = 4'b0100;  
        core[4] = 4'b0101;  
        core[5] = 4'b0110;  
        core[6] = 4'b0111;  
        core[7] = 4'b1000;  
    end  
end
```



```

initial // Test stimulus
begin
    #0 Data = 4'b0000;
    #50 Data = 4'b0001;
    #50 Data = 4'b0010;
    #50 Data = 4'b0011;

    #50 Data = 4'b0100;
    #50 Data = 4'b0101;
    #50 Data = 4'b0110;
    #50 Data = 4'b0111;

    #50 Data = 4'b1000;
    #50 Data = 4'b1001;
    #50 Data = 4'b1010;
    #50 Data = 4'b1011;

    #50 Data = 4'b1100;
    #50 Data = 4'b1101;
    #50 Data = 4'b1110;
    #50 Data = 4'b1111;

    #200 $stop;
end

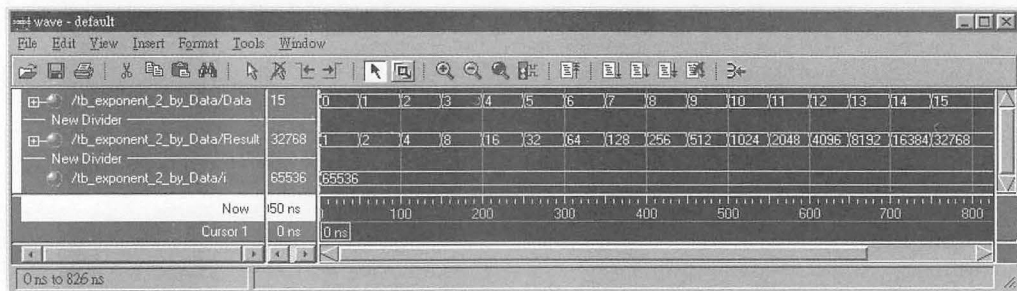
endmodule

// exponent_2_by_Data.v
module exponent_2_by_Data(Data, Result);
    input [3:0] Data; // 0 ~ 15
    output [15:0] Result; // 2 ** 15 = 32768, needing 16 Bits
    wire [15:0] Result;

    assign Result = 2 ** Data;

endmodule

```





13.9 Re-entrant Task and Recursive Function

Verilog 2001 增加了 “Re-entrant task” 以及 “Recursive” 允許本身叫用本身的 “遞迴函數”。

不同於 Verilog 1995 的地方是：Verilog 2001 每一次叫用都會配置一塊獨立的記憶體用於儲存該 task/function 中所需用到的記憶體空間。當然，二個或以上的地方同時叫用該 task/function 時，所得到的記憶體空間是不相同的。

將來 Verilog 2001 的 “Re-entrant Task” 及 “Recursive Function” 是可以作為電路合成的。

新增加的 Verilog 保留字：**automatic**。

可惜的是 ModelSim SE 5.7c 並沒有支援 “Automatic tasks” 以及 “Automatic functions” 這二種寫法。

A Re-entrant Task 的例子

```

task automatic auto_task;
  input call;
  if (call)
    auto_task(0); // automatic task
endtask

```

A Recursive Function 的例子

```

module Recursive (data_in, N_Fact);
  input  [ 3:0] data_in; // 0 ~ 15
  output [31:0] N_Fact;  // 1 * 2 * 3 * 4 * ... * 15 = 2004310016,
                        // needing 31 Bits
  wire  [31:0] N_Fact;

  // ModelSim SE 5.7c 不支援 Automatic functions

```

```
function automatic [63:0] factorial;
  input [31:0] n;
  if (n == 1)
    factorial = 1;
  else
    factorial = n * factorial(n-1); // automatic function
endfunction

assign N_Fact = factorial( data_in );

endmodule
```



13.10 Comma-separated Sensitivity List

Verilog 2001 增加了另一種用於描寫 sensitivity list (感測列) 的寫法，Verilog 1995 是用 or 這個關鍵字來區分不同的訊號，而 Verilog 2001 則是可以用像 VHDL 一樣用，(逗號) 來區分不同的訊號。

Verilog 2001 的 “Comma-separated Sensitivity List” 是可以作為電路合成的。

Synplify 7.1 版以及 Synplify Pro 7.1 版，有支援 “Comma-separated Sensitivity List”。

SIMUCAD 的 Silos Verilog Logic Simulator，有支援 “Comma-separated Sensitivity List”。

ModelSim SE 5.5 之後的版本就有支援 “Comma-separated Sensitivity List” 了。

Comma-separated Sensitivity List 的例子

```
module Comma_Separated (a, b, c, d, sel, y1, y2, y3);
  input [ 3:0] a, b, c, d;
```

```

input  [ 1:0] sel;
output [ 3:0] y1, y2, y3;
reg    [ 3:0] y1, y2, y3;

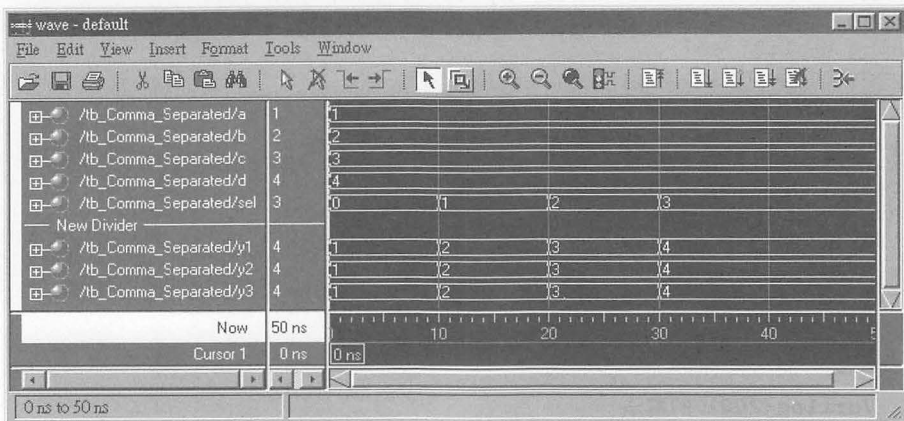
// Verilog-1995 的寫法
always @(sel or a or b or c or d)
  case (sel)
    2'b00: y1 = a;
    2'b01: y1 = b;
    2'b10: y1 = c;
    2'b11: y1 = d;
  endcase

// Verilog-2001 的寫法 1
always @(sel, a, b, c, d)
  case (sel)
    2'b00: y2 = a;
    2'b01: y2 = b;
    2'b10: y2 = c;
    2'b11: y2 = d;
  endcase

// Verilog-2001 的寫法 2
always @(sel, a or b or c, d) // , 與 or 混合使用
  case (sel)
    2'b00: y3 = a;
    2'b01: y3 = b;
    2'b10: y3 = c;
    2'b11: y3 = d;
  endcase

endmodule

```





13.11 Combinational Logic Sensitivity

Verilog 2001 在 sensitivity list (感測列) 部份又增加了 “wild-card” 的功能，@* 這個 wildcard 字眼會自動地將 always 區中找到該寫在 sensitivity list (感測列) 的訊號。

Verilog 2001 的 “Combinational Logic Sensitivity” 是可以作為電路合成的。

Quartus II 2.1 版，有支援 “Combinational Logic Sensitivity”。

Synplify 7.1 版以及 Synplify Pro 7.1 版，有支援 “Combinational Logic Sensitivity”。

SIMUCAD 的 Silos Verilog Logic Simulator，有支援 “Combinational Logic Sensitivity”。

ModelSim SE 5.7 之後的版本才有支援 “Combinational Logic Sensitivity”。

Combinational Logic Sensitivity 的例子 1

```
module Comb_Sensitivity_1 (a, b, c, d, sel, y1, y2);
  input  [ 3:0] a, b, c, d;
  input  [ 1:0] sel;
  output [ 3:0] y1, y2;
  reg     [ 3:0] y1, y2;

  // Verilog-1995 的寫法
  always @(sel or a or b or c or d)
    case (sel)
      2'b00: y1 = a;
      2'b01: y1 = b;
      2'b10: y1 = c;
      2'b11: y1 = d;
    endcase

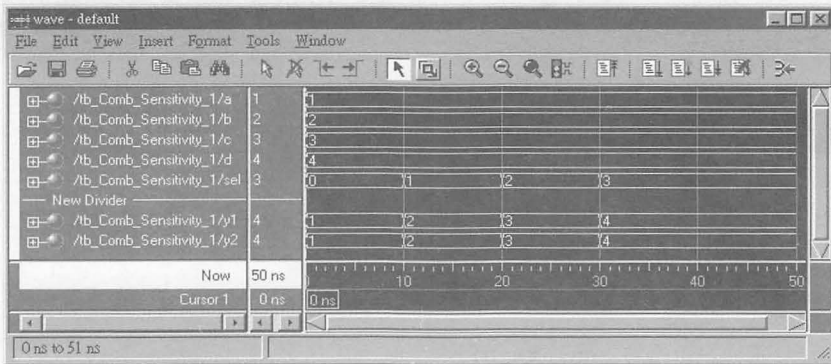
  // Verilog-2001 的寫法
```

```

always @* // 相當於 (sel or a or b or c or d)
case (sel)
  2'b00: y2 = a;
  2'b01: y2 = b;
  2'b10: y2 = c;
  2'b11: y2 = d;
endcase

endmodule

```



Combinational Logic Sensitivity 的例子 2

```

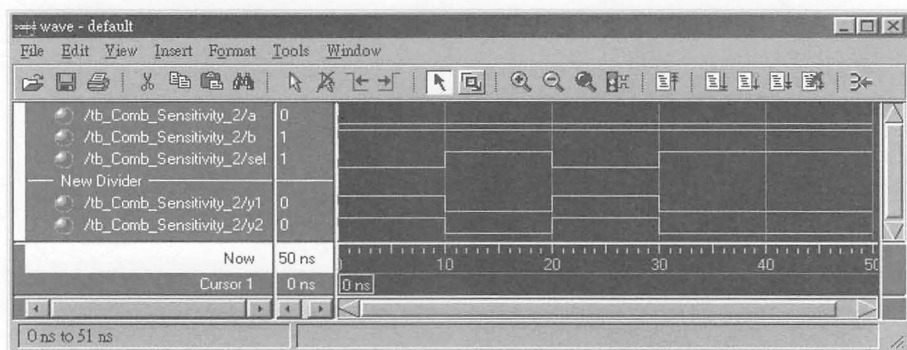
module Comb_Sensitivity_2 (a, b, sel, y1, y2);
  input a, b;
  input sel;
  output y1, y2;
  reg y1, y2;

  // Verilog-1995 的寫法
  always @(a, b, sel)
    if (sel)
      y1 = a;
    else
      y1 = b;

  // Verilog-2001 的寫法
  always @*
    if (sel)
      y2 = a;
    else
      y2 = b;

endmodule

```



Combinational Logic Sensitivity 的例子 3

```

module Comb_Sensitivity_3 (en, rw_n, addr, data, data_o1, data_o2);
  parameter ADDR_SIZE = 8;
  parameter DATA_SIZE = 4;
  parameter MEM_DEPTH = 1 << ADDR_SIZE;

  input          en, rw_n;
  input [ADDR_SIZE-1:0] addr;
  input [DATA_SIZE-1:0] data;
  output [DATA_SIZE-1:0] data_o1;
  output [DATA_SIZE-1:0] data_o2;

  reg [DATA_SIZE-1 : 0] mem_1 [0 : MEM_DEPTH-1];
  reg [DATA_SIZE-1 : 0] mem_2 [0 : MEM_DEPTH-1];

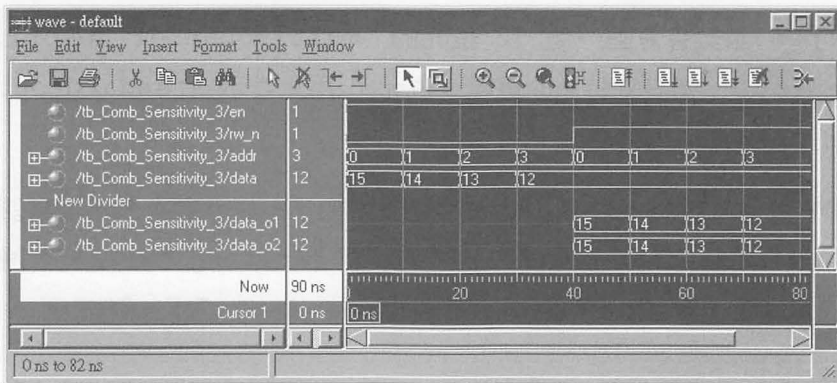
  // Verilog-1995 的寫法
  always @(rw_n, en, addr, data)
    if (!rw_n && en)
      mem_1[addr] = data; // 寫入

  // Verilog-2001 的寫法
  always @*
    if (!rw_n && en)
      mem_2[addr] = data; // 寫入

  // 讀取
  assign data_o1 = (rw_n && en) ? mem_1[addr] : {DATA_SIZE{1'bz}};

  assign data_o2 = (rw_n && en) ? mem_2[addr] : {DATA_SIZE{1'bz}};

endmodule
  
```



13.12 Enhanced File I/O

Verilog 1995 對於檔案的處理，最多只能同時開啓 31 個檔案，而且也只能處理文字檔案，比較複雜的檔案處理得藉由 Programming Language Interface (PLI) 來達成。

而 Verilog 2001 則是最多能同時開啓 2^{30} 個檔案，而且也增加了存取二進制檔案的功能。

新增加的檔案處理函數：`$ferror`、`$fgetc`、`$fgets`、`$fflush`、`$fread`、`$fscanf`、`$fseek`、`$ftel`、`$rewind` 以及 `$ungetc`。

新增加的字串處理函數：`$sformat`、`$swrite`、`$swriteb`、`$swriteh` 以及 `$swriteo`。

Verilog 2001 的“Enhanced File I/O”是**不能**合成的 VHDL 語法。

SIMUCAD 的 Silos Verilog Logic Simulator 有支援“Enhanced File I/O”。

ModelSim SE 5.5 之後的版本就有支援“Enhanced File I/O”了。

\$fopen 函數會傳回一個 32 位元的 “multi-channel descriptor” (mcd)，每個位元都有它特別的意義，位元 0 用來表示 standard out，也就是螢幕。

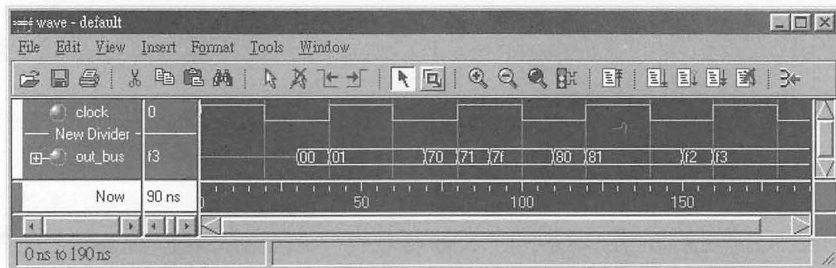
在 Verilog 2001 中，\$fopen 函數增加了一個參數，表示開啓該檔案是用來 “讀取” (reading, “r”)、 “寫入” (writing “w”)、 “讀取與寫入” (update) 以及增加資料到檔案末端 (append, “a”)。例如：

```
mcd = $fopen{"file_name"};
fd = $fopen{"file_name", <type>;
```

其他詳細的使用方式請參考：<http://chris.spear.net/pli/fileio.htm> **Reading and writing files from Verilog models。**

File I/O 的例子

```
reg [7:0] out_bus;
integer f1, f2;
initial
begin
    f1 = $fopen("out_file_1.dat" );
    f2 = $fopen("out_file_2.dat" );
    forever @(out_bus)
        if ( out_bus >= 8'h80 ) // 如果 out_bus >= 128
            $fstrobe(f1 | f2 | 1, "fstrobe OUT_BUS at time %t", $realtime);
            // f1 : 輸出到檔案 out_file_1.dat
            // f2 : 輸出到檔案 out_file_2.dat
end
// 1 : standard out 也就是螢幕上
```





13.13 Automatic Width Extension Past 32 bits

當“左邊的資料寬度”大於“右邊的資料寬度”時，Verilog 1995 “左邊的資料”多出來的高位元部份補‘0’。對於未指定整數(unsized integer)的資料寬度，內定是 32 個位元，因此如果給超出 32 位元時，則必須用**指定長度給值**的方式來做。而在 Verilog 2001，這個問題就不存在了。

Verilog 2001 的“Automatic Width Extension Past 32 bits”是否能夠作為電路合成，得看 EDA tool 廠商要不要設計而定。

網頁上寫：http://www.model.com/products/verilog_ad/verilog_2001.asp ModelSim SE 5.7 之後的版本才有支援“Automatic Width Extension Past 32 bits”。不過實驗的結果，ModelSim SE 5.7c 仍然沒有辦法得到“Automatic Width Extension Past 32 bits”的作用。

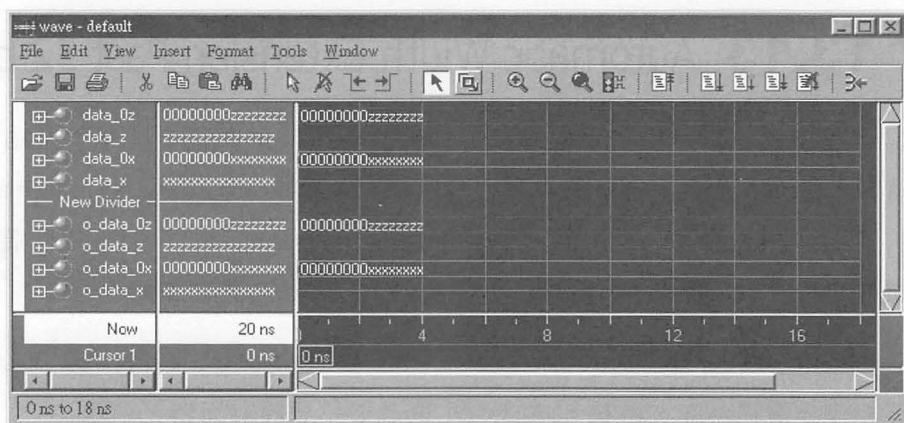
Automatic Width Extension Past 32 bits 的例子

【Verilog 1995 的寫法】

```
parameter WIDTH = 64;
reg [WIDTH-1:0] data;
data = 'bz; // data 的值會等於 'h00000000zzzzzzzz
// 必須改成底下指定長度給值的方式
data = 64'bz; // data 的值會等於 'hzzzzzzzzzzzzzzzzzzzz
```

【Verilog 2001 的寫法】

```
parameter WIDTH = 64;
reg [WIDTH-1:0] data1, data2;
data1 = 'bz; // data1 的值會等於 'hzzzzzzzzzzzzzzzzzzzz
data2 = 'bx; // data1 的值會等於 'hxxxxxxxxxxxxxxxx
```



13.14 Default Net with Continuous Assign

在 Verilog 1995 時，指定運算式給左手邊 (left-hand side) 的 reg 或是 wire (除了 out/inout port 以外) 必都得事先宣告。

而在 Verilog 2001，如果指定運算式給左手邊 (left-hand side) 的 reg 或是 wire (除了 output/inout port 以外) 沒有事先宣告的話，內定為 1 個 bit 的 wire。

Verilog 2001 的 “Default Net with Continuous Assign” 是否能夠作為電路合成，得看 EDA tool 廠商要不要設計而定。

Default Net with Continuous Assign 的例子 1

【Verilog 1995 的寫法】

```
module mult32(a, b, y);
  input  [31:0] a, b;
  output [63:0] y;

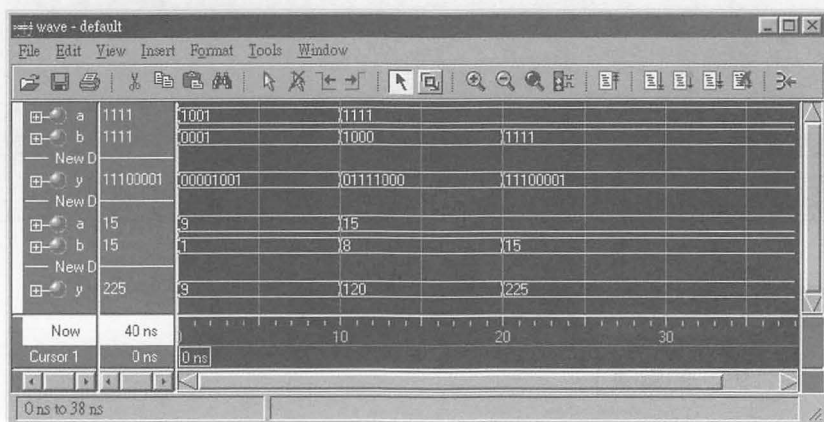
  assign y = a * b;    // y 雖沒有宣告，但它是 output port
```

```
// 錯誤：因為 eq 沒有宣告，而且也不是 output/inout port
assign eq = (a == b);
endmodule
```

【Verilog 2001 的寫法】

```
module mult32(a, b, y);
  input  [31:0] a, b;
  output [63:0] y;

  assign y = a * b;    // y 雖沒有宣告，但它是 output port
  // 正確：eq 沒有宣告，而且也不是 output/inout port，所以內定為 1 個 bit 的 wire
  assign eq = (a == b);
endmodule
```



Default Net with Continuous Assign 的例子 2

【Verilog 1995 的寫法】

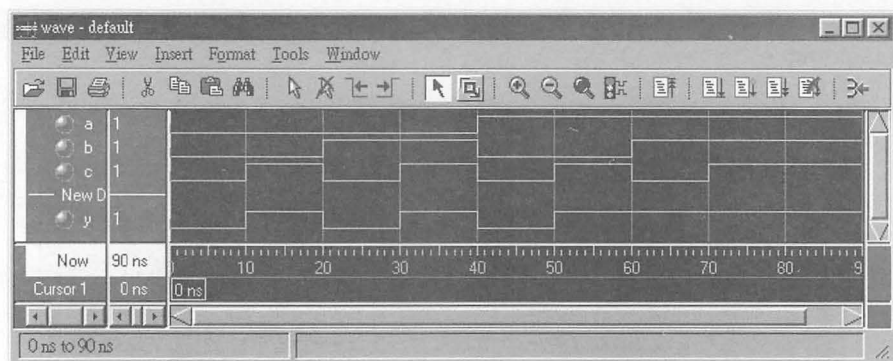
```
module and_or(a, b, c, y);
  input  a, b, c;
  output y;
  wire n1; // 需要宣告 wire n1;

  assign n1 = a & b;
  assign y = n1 | c;
endmodule
```

【Verilog 2001 的寫法】

```
module and_or(a, b, c, y);
    input  a, b, c;
    output y;

    assign n1 = a & b; // 不需要宣告 wire n1;
    assign y = n1 | c;
endmodule
```



Default Net with Continuous Assign 的例子 3

【Verilog 1995 的寫法】

```
module muxff_1995(clk, rst_n, ce, d, q);
    input d, clk, ce, rst_n;
    output q;
    reg q;
    wire y;

    assign y = ce ? d : q;

    always @(posedge clk or negedge rst_n)
        if (!rst_n)
            q <= 0;
        else
            q <= y;
endmodule
```

【Verilog 2001 的寫法】

```

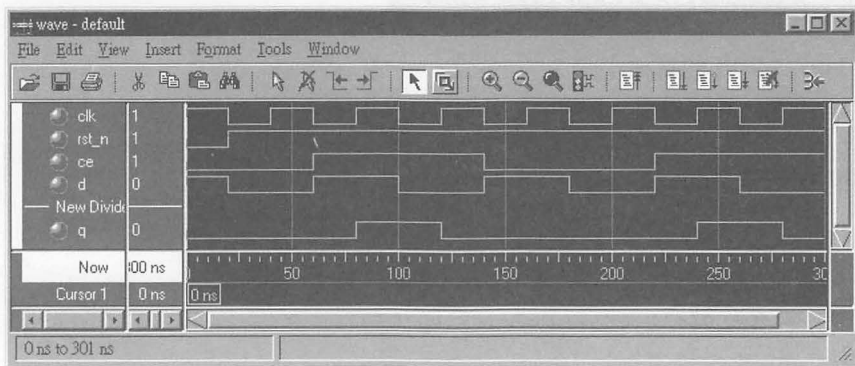
module muxff_2001(clk, rst_n, ce, d, q);
  input d, clk, ce, rst_n;
  output q;
  reg q;

  wire y = ce ? d : q; // 宣告 wire y 同時也令 y = cd ? d : q;

  // 可以只寫成：
  assign y = ce ? d : q; // 不用事先宣告 wire y;

  always @(posedge clk or negedge rst_n)
    if (!rst_n)
      q <= 0;
    else
      q <= y;
endmodule

```

**13.15** Disable Default Net Declaration

在 Verilog 1995 時，對於沒有宣告的訊號，內定都把它視為 wire。而您也可以藉由 `default_nettype <data_type> 這個 compiler directive，讓您將內定為 wire 改成其它種類的 net 型態，例如：wire、wor、wand、tri、tri0、tri1、triand、trior 或者是 trireg。

而在 Verilog 2001，則提供了 ``default_nettype none` 這個 compiler directive，讓您中止使用內定 `net` 型態，目的在於防止出現因為訊號名稱出錯而造成難以除錯的問題。

Verilog 2001 的 “Disable Default Net Declaration” 是否能夠作為電路合成，得看 EDA tool 廠商要不要設計而定，因為 ``default_nettype` 是一個 compiler directive。

ModelSim SE 5.7 之後的版本才有支援 “Disable Default Net Declaration”。

新增加的 Verilog 保留字：`none`。

Disable Default Net Declaration 的例子

【Verilog 1995 的寫法】

Verilog-1995 would compile without an error, requiring debugging in simulation

```
module bad_chip(n0, n1, o1);
  input  [7:0] n0;
  wire   [7:0] n0;
  input  [7:0] n1;
  wire   [7:0] n1;
  output [7:0] o1;
  wire   [7:0] o1; // 是 wire 不是 reg

  assign o1 = n0 & n1;
endmodule
```

【Verilog 2001 的寫法】

Verilog-2001 will report an error on the undefined signals

```
`default_nettype none
module bad_chip(input wire [7:0] n0,
```



```

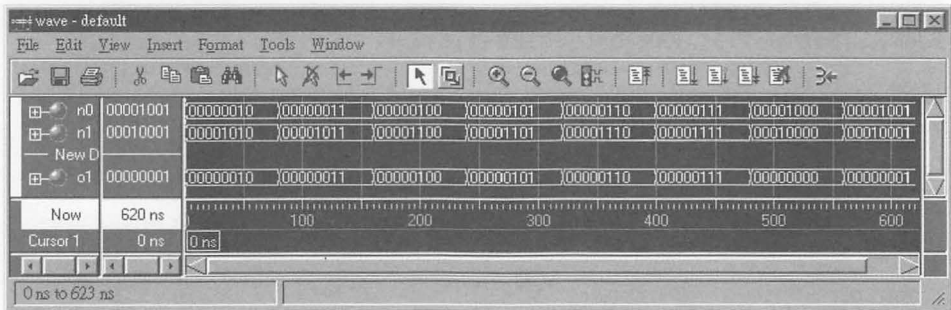
        input wire [7:0] n1,
        output wire [7:0] o1 );

// 如果有 `default_nettype none, compile 會告知有錯誤
assign o1 = n0 & n1; // o1 打錯字 (打成小寫的 L)

// 如果沒有 `default_nettype none, compile 是不會告知有錯誤的
assign o1 = n0 & n1; // there are 3 type's in this line

endmodule

```



13.16 Explicit In-line Parameter Passing

在 Verilog 2001，增加了引用有 parameter 的 module 時，可用 .parameter_name(value) 的格式來傳遞參數。目的在於不必考慮 parameter 的順序，比較不容易出錯，以及對於日後的維護也比較方便。

Verilog 2001 的 “Explicit In-line Parameter Passing” 是可以作為電路合成的。

SIMUCAD 的 Silos Verilog Logic Simulator，有支援 “Explicit In-line Parameter Passing”。

ModelSim SE 5.6 之後的版本才有支援 “Explicit In-line Parameter Passing”。

Explicit In-line Parameter Passing 的例子

```

module RAM (CS, RW, Address, Data_In, Data_Out);
    parameter ADDRESS_SIZE = 8;
    parameter WIDTH = 8;
    parameter SIZE = 256;

    input CS, RW;
    input [ADDRESS_SIZE-1:0] Address;
    input [WIDTH-1:0] Data_In;
    output [WIDTH-1:0] Data_Out;
    reg [WIDTH-1:0] Data_Out;
    reg [WIDTH-1:0] RAM_Data[SIZE-1:0];

    always @(negedge CS)
    begin // 負緣觸發，選擇到此晶片
        if (RW == 1'b0) // 讀取
            begin
                Data_Out = RAM_Data[Address[2:0]];
            end
        else
            begin
                if (RW == 1'b1) // 寫入
                    begin
                        RAM_Data[Address[2:0]] = Data_In;
                    end
                else
                    Data_Out = 3'bz; // 高阻抗 (High Impedance)
                end
            end
        end
    end

endmodule

```

【Verilog 1995 的寫法】

```

module tb_RAM;
    reg CS, RW;
    reg [ADDRESS_SIZE-1:0] Address;
    reg [WIDTH-1:0] Data_In;
    wire [WIDTH-1:0] Data_Out;

    RAM #(ADDRESS_SIZE, WIDTH, SIZE)
        i_RAM (.CS(CS), .RW(RW), .Address(Address),
              .Data_In(Data_In), .Data_Out(Data_Out));

    ...
endmodule

```

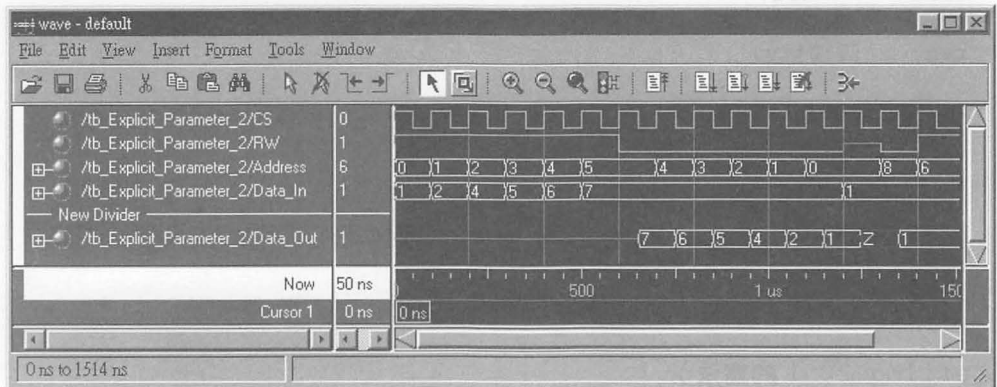
【Verilog 2001 的寫法】

```

module tb_RAM;
  reg CS, RW;
  reg [ADDRESS_SIZE-1:0] Address;
  reg [WIDTH-1:0] Data_In;
  wire [WIDTH-1:0] Data_Out;

  RAM # ( .SIZE(SIZE),
          .ADDRESS_SIZE(ADDRESS_SIZE),
          .WIDTH(WIDTH) )
    i_RAM ( .CS(CS), .RW(RW), .Address(Address),
            .Data_In(Data_In), .Data_Out(Data_Out));
  ...
endmodule

```

**13.17** Combined Port/Data Type Declaration

在 Verilog 2001，允許將 port 的宣告與資料型態寫成同一道敘述。

Verilog 2001 的“Combined Port/Data Type Declaration”是可以作為電路合成的。

Synplify 7.1 版以及 Synplify Pro 7.1 版，有支援“Combined

Port/Data Type Declaration”。

ModelSim SE 5.5 之後的版本就有支援 “Combined Port/Data Type Declaration” 了。

Combined Port / Data Type Declaration 的例子

【Verilog 1995 的寫法】

```
module mux8 (sel, a, b, y);
    input      sel;    // input 搭配 wire, 分成 2 道敘述寫
    input  [7:0] a, b; // input 搭配 wire, 分成 2 道敘述寫
    output [7:0] y;    // output 搭配 reg, 分成 2 道敘述寫
    wire      sel;
    wire  [7:0] a, b;
    reg  [7:0] y;

    always @(sel, a, b)
        if (sel)
            y = a;
        else
            y = b;

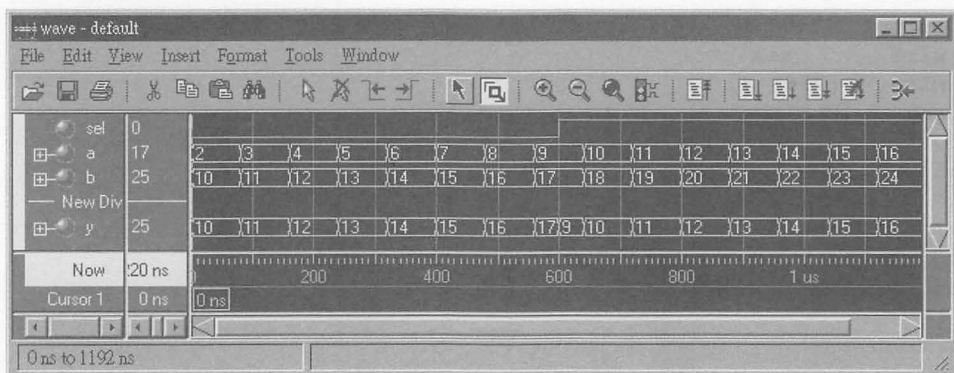
endmodule
```

【Verilog 2001 的寫法】

```
module mux8(a, b, en, y);
    input wire  sel; // input 搭配 wire, 寫成同 1 道敘述
    input wire [7:0] a, b; // input 搭配 wire, 寫成同 1 道敘述
    output reg  [7:0] y; // output 搭配 reg, 寫成同 1 道敘述

    always @(sel, a, b)
        if (sel)
            y = a;
        else
            y = b;

endmodule
```



13.18 ANSI-style Port List

在 Verilog 2001，增加可以將 module、task 及 function 的 input 以及 output port 寫成和 ANSI C 風格的寫法。

Verilog 2001 的“ANSI-style Port List”是可作為電路合成的。

Synplify 7.1 版以及 Synplify Pro 7.1 版，有支援“ANSI-style Port List”。

SIMUCAD 的 Silos Verilog Logic Simulator，有支援“ANSI-style Port List”。

ModelSim SE 5.6 之後的版本才有支援“ANSI-style Port List”。

ANSI-style module Port List 的例子 1

【Verilog 1995 的寫法】

```
module mux8(sel, a, b, y);
    input      sel; // input 搭配 wire, 分成 2 道敘述寫
    input  [7:0] a, b; // input 搭配 wire, 分成 2 道敘述寫
```

```
output [7:0] y; // output 搭配 reg, 分成 2 道敘述寫
wire      sel;
wire      [7:0] a, b;
reg       [7:0] y;

always @(sel, a, b)
    if (sel)
        y = a;
    else
        y = b;

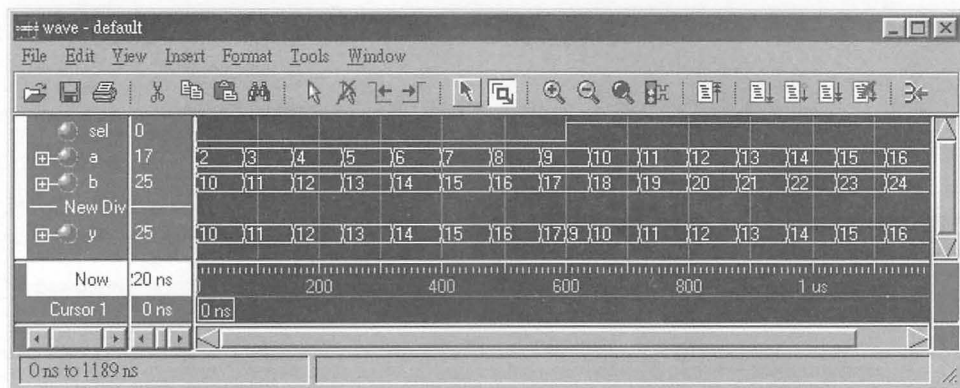
endmodule
```

【Verilog 2001 的寫法】

```
module mux8(input wire sel,
            input wire [7:0] a,
            input wire [7:0] b,
            output reg [7:0] y);

    always @(sel, a, b)
        if (sel)
            y = a;
        else
            y = b;

endmodule
```



ANSI-style module Port List 的例子 2

```
module ansi_port_list( input wire      en,
                      input wire [7:0] a,
                      output reg  [7:0] y1,
                      output reg  [7:0] y2);

  always @*
  begin
    if (en)
    begin
      y1 = ~a;
      y2 = ~a;
    end
    else
    begin
      y1 = ~a;
      y2 = ~a;
    end
  end
end
```

ANSI-style module Port List 的例子 3

【Verilog 1995 的寫法】

```
module dff_async_rst (clk, rst_n, d, q);
  input  clk, rst_n, d;
  output q;
  reg    q;

  always @(posedge clk or negedge rst_n)
    if (!rst_n)
      q <= 1'b0;
    else
      q <= d;
endmodule
```

【Verilog 2001 的寫法】

```
module dff_async_rst (input  clk, rst_n, d,
                     output reg q);

  always @(posedge clk or negedge rst_n)
    if (!rst_n)
```

```

        q <= 1'b0;
    else
        q <= d;
    endmodule

```

Verilog-2001 ANSI-style task 的例子 1

```

module my_module(...);
...
    always @(posedge clock)
    begin
        SendData(data, dout);
        ...
    end

    task SendData(input [31:0] data_bus,
                  output [31:0] data_out);
        begin
            #3.2 data_out = data_bus;
        end
    endtask
endmodule

```

Verilog-2001 ANSI-style task 的例子 2

```

task ansi_port_task (
    input [7:0] a,
    input [7:0] b,
    input opcode,
    output [8:0] result);

    result = (opcode == 1) ? (a + b) : (a - b);
endtask

```

Verilog-2001 ANSI-style function 的例子

```

function [8:0] add_or_sub (
    input [7:0] a,
    input [7:0] b,
    input opcode);

    add_or_sub = (opcode == 1) ? (a + b) : (a - b);
endfunction

```



13.19 Reg Declaration With Initialization

在 Verilog 2001，增加可以在宣告 reg 的同時給定“初始值”，給定初始值是在時間為 0 (time-step zero) 的時候，和在 initial 一開始時給值的寫法是一樣的。

Verilog 2001 的“Reg Declaration With Initialization”只能用在寫 test bench (測試平台程式) 時使用的，它是不能合成的 VHDL 語法。

ModelSim SE 5.5 之後的版本就有支援“Reg Declaration With Initialization”了。

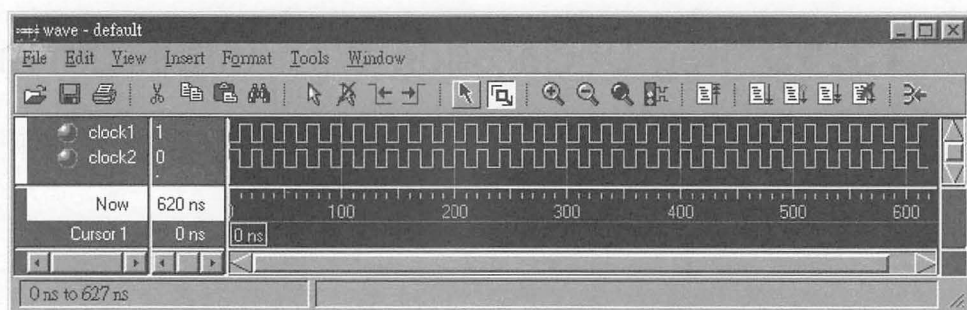
Reg Declaration With Initialization 的例子

【Verilog 1995 的寫法】

```
reg clock1;  
initial clock1 = 0;  
  
always  
begin  
    #10 clock1 = ~clock1;  
end
```

【Verilog 2001 的寫法】

```
reg clock2 = 1;  
  
always  
begin  
    #10 clock2 = ~clock2;  
end
```

13.20 "Register" Changed To "Variable"

Verilog 2001 將 register 這個字眼改成 variable 的用途，起初在 1984 Verilog 手冊中是使用 register 這個 class 去描述資料型態中的：reg(無號數變數)、integer(有號數變數)，以及 real(雙精確度變數)。實際上 Verilog 的 register 宣告並不代表它就會是一個硬體的 register。



13.21 Enhanced PLA Modeling

在 Verilog 1995 中，傳遞給 PLA system task 只能是常數(scalar)。

在 Verilog 2001 中，傳遞給 PLA system task 也可以是陣列(vector) (\$asyn\$or\$array、\$asyn\$and\$array...等等)。



13.22 Accurate BNF, with Subsection

在 Verilog 1995 BNF(Backus-Naur Form)中，有錯誤不一致的地方，Verilog 2001 則改正了這些問題。



13.23 Enhanced Conditional Compilation

在 Verilog 1995 中，提供的條件式編譯(conditional compilation)敘述是比較有限的，只有`ifdef、`else、`endif 以及`undef。

而在 Verilog 2001 中，則增加了`ifndef 以及`elsif 這二個條件式編譯(conditional compilation)敘述。

Verilog 2001 的“Enhanced Conditional Compilation”是可以作為電路合成的。

Quartus II 2.1 版，有支援`ifdef 以及`elsif 這二個 compiler directive。

ModelSim SE 5.5 之後的版本就有支援`ifdef 以及`elsif 這二個 compiler directive。

新增加的 compiler directive：`ifndef 以及`elsif。

Conditional Compilation 的例子 1

【Verilog 1995 的例子】

```
`ifdef <macro_name>
  <verilog_source_code>
`else
  <verilog_source_code>
`endif
```

【Verilog 2001 的例子】

```
`ifdef RTL_TEST
  Design_RTL ul(...);
`elsif GATE
  Design_Gate ul(...);
```

```
`else
  initial $display( "ERROR: neither RTL or GATE model instantiated ");
`endif
```

Conditional Compilation 的例子 2

【Verilog 1995 的例子】

```
`ifdef SYNTHESIS
  // Do nothing
`else
  initial $display("Running RTL Model");
`endif
```

【Verilog 2001 的例子】

```
`ifndef SYNTHESIS
  initial $display("Running RTL Model");
`endif
```

【Enhanced Conditional Compilation 的實例】

```
// Mux1.v
module Mux_1 (sel, a, b, y);
  input      sel;
  input  [7:0] a, b;
  output  [7:0] y;

  wire      sel;
  wire  [7:0] a, b;
  reg   [7:0] y;

  always @(sel, a, b)
    if (sel)
      y = a;
    else
      y = b;

endmodule
```

```
// Mux2.v
module Mux_2 (input wire sel,
              input wire [7:0] a,
              input wire [7:0] b,
              output reg [7:0] y);

    always @(sel, a, b)
        if (sel)
            y = a;
        else
            y = b;

endmodule

// tb_Mux.v
// `define Verilog_1995
`define Verilog_2001

// `define SYNTHESIS

module tb_Mux;

    reg sel;
    reg [7:0] a, b;
    wire [7:0] y;

    `ifdef Verilog_1995
        Mux_1 i_Mux_1 ( sel, a, b, y );
    `elsif Verilog_2001
        Mux_2 i_Mux_2 ( sel, a, b, y );
    `else
        initial $display( "ERROR: neither Mux_1 or Mux_2 model
            instantiated" );
    `endif

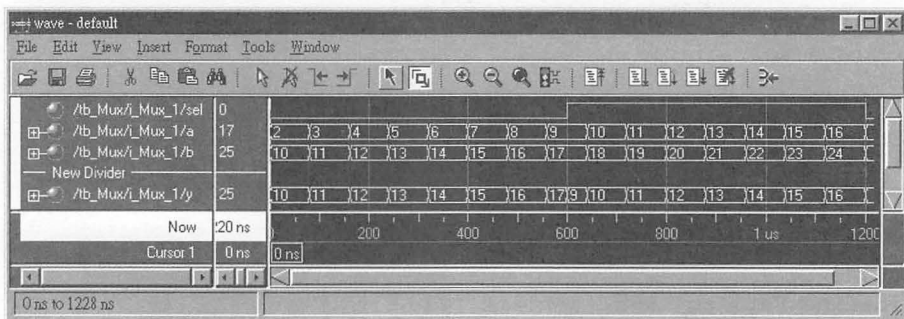
    `ifdef SYNTHESIS
        // Do nothing
    `else
        initial $display("Running RTL Model");
    `endif

    `ifndef SYNTHESIS
        initial $display("Running RTL Model");
    `endif

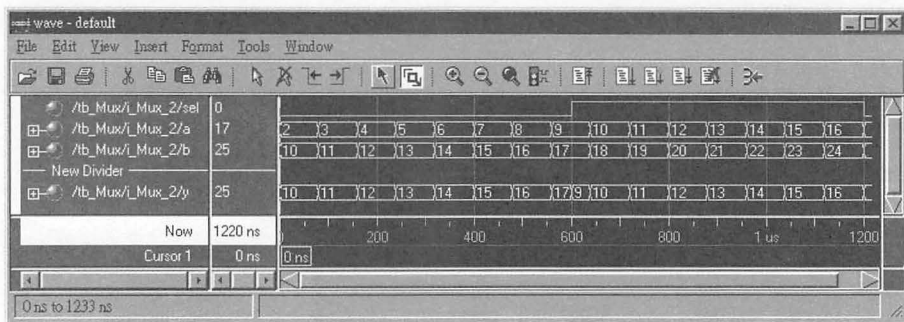
    ...

endmodule
```

當有 ``define Verilog_1995` 或同時有 ``define Verilog_1995` 與 ``define Verilog_2001` 時，會用 `Mux_1 i_Mux_1(sel, a, b, y);`;



當有 ``define Verilog_2001` 時，會用 `Mux_2 i_Mux_2(sel, a, b, y);`;



當沒有 ``define Verilog_1995` 或 ``define Verilog_2001` 時，ModelSim 會用 `$display` 函數顯示 `ERROR: neither Mux_1 or Mux_2 model instantiated` 來讓您知道。



13.24 File and Line Compiler Directive

在 Verilog 2001 中，則增加了 ``line` 以及 ``file` 這二個 compiler directive 用於記錄原始的 Verilog 檔案位置。

Verilog 2001 的 “File and Line Compiler Directive” 是可以作為電路合成的。

Quartus II 2.1 版，有支援 ``line` 以及 ``file` 這二個 compiler directive。

ModelSim SE 5.5 之後的版本就有支援 ``line` 以及 ``file` 這二個 compiler directive。

新增加的 compiler directive: ``line` 以及 ``file`。

13.25 Attribute

在 Verilog 2001 中，則增加了 attribute 這個特性(property)，它是一個標準的方法讓 tool 能夠為 module 指定資訊。

有了它就可以不用像以前一樣將 EDA tool 特定的 command 寫在註解(comment)中了，而 EDA tool 更可以自訂專用的 attribute。

Verilog 2001 的 “Attribute” 是可以作為電路合成的。

Quartus II 2.1 版才有支援 “Attribute”，之前的版本沒有支援。

SIMUCAD 的 Silos Verilog Logic Simulator，有支援 “Attribute”。

ModelSim SE 5.5 之後的版本就有支援 “Attribute” 了。可惜的是實作不出來，不成功。

新增加的 token: `(*` 以及 `*)`。

『Verilog 1995 的語法』

```
// synthesis <attribute> { <value> }
```

或者是

```
/* synthesis <attribute> { <value> } */
```

『Verilog 2001 的語法』

```
(* <attribute> { = <value> } *)
```

Attribute 的例子 1

【Verilog 1995 的寫法】

```
case (1'b1) /* synopsys parallel_case */ // 1-Hot FSM
```

【Verilog 2001 的寫法】

```
(* rtl_synthesis, parallel_case *) case (1'b1) // 1-Hot FSM  
實作不出來，不成功。
```

Attribute 的例子 2

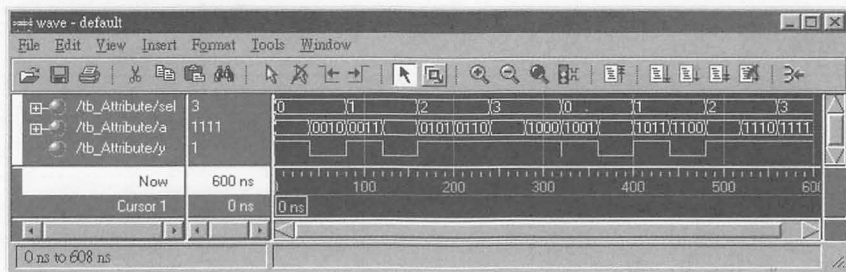
【Verilog 1995 的寫法】

```
module test_case(sel, a, y);  
  input  [1:0] sel;  
  input  [3:0] a;  
  output      y;  
  reg         y;  
  
  always @(sel or a)  
    case (sel) // synthesis full_case
```

```

2'b00 : y = a[0];
2'b01 : y = a[1];
2'b10 : y = a[2];
endcase
endmodule

```



【Verilog 2001 的寫法】

```

module test_case(sel, a, y);
  input [1:0] sel;
  input [3:0] a;
  output      y;
  reg         y;

  always @(sel or a)
  (* full_case *) case (sel) // 實作不出來，不成功
    2'b00 : y = a[0];
    2'b01 : y = a[1];
    2'b10 : y = a[2];
  endcase
endmodule

```

Attribute 的例子 3

【Verilog 1995 的寫法】

```

module test_casez(sel, a, b, c);
  input [2:0] sel;
  output      a, b, c;
  reg         a, b, c;

  always @(sel)

```



```
begin
  {a, b, c} = 3'b0;
  casez (sel) // synthesis parallel_case
    3'b1?? : a = 1'b1;
    3'b?1? : b = 1'b1;
    3'b??1 : c = 1'b1;
  endcase
end
endmodule
```

【Verilog 2001 的寫法】

```
module test_casez(sel, a, b, c);
  input [2:0] sel;
  output      a, b, c;
  reg         a, b, c;

  always @(sel)
  begin
    {a, b, c} = 3'b0;
    (* parallel_case *) casez (sel) // 實作不出來，不成功
      3'b1?? : a = 1'b1;
      3'b?1? : b = 1'b1;
      3'b??1 : c = 1'b1;
    endcase
  end
endmodule
```



13.26 Standard Random Number Generator

在 Verilog 2001 中，增加了 C 語言中的 **\$random** 函數，它特別強調的是任何一個電路模擬都能夠產生相同的亂數序列 (random number sequence)，前提是亂數種子 (seed) 一樣。

Verilog 2001 的 “Standard Random Number Generator” 是否能夠作為電路合成，得看 EDA tool 廠商要不要設計而定。

ModelSim SE 5.5 之後的版本就有支援 “Standard Random Number

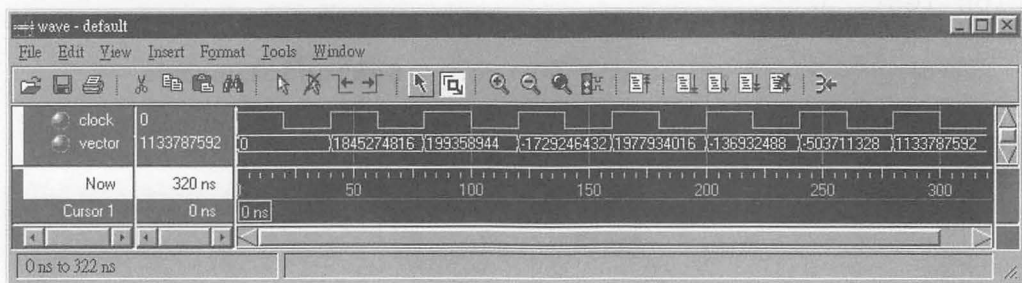
Generator”了。

Standard Random Number Generator 的例子

```
reg clock;
initial clock = 1;
always
begin
    #10 clock = ~clock;
end

integer seed, vector;
initial seed = 1;

always @(posedge clock)
begin
    // 取得目前電路的模擬時間 $time
    // 缺點：每次電路開始執行時產生的 "亂數值" 都相同
    vector = $time * $random(seed);
end
```



13.27 Enhanced Invocation Option Test

在 Verilog 1995 中，有 true/false 測試是否電路模擬時所傳遞的參數有給特定的選項 **\$test\$plusargs**。

在 Verilog 2001 中，則增加了能夠讀取電路模擬時所傳遞的參數，語法如下：**\$value\$plusargs("option=%<format>", <variable>);**

其中 %<format> 可以是：

- %b 二進制值
- %o 八進制值
- %d 十進制值
- %h 十六進制值
- %e 實數 exponential format
- %f 實數 decimal format
- %g 實數 shortest format
- %s 字串

Verilog 2001 的 “Enhanced Invocation Option Test” 是電路模擬時的所傳遞的參數，和是否能夠作為電路合成這個特性無關。

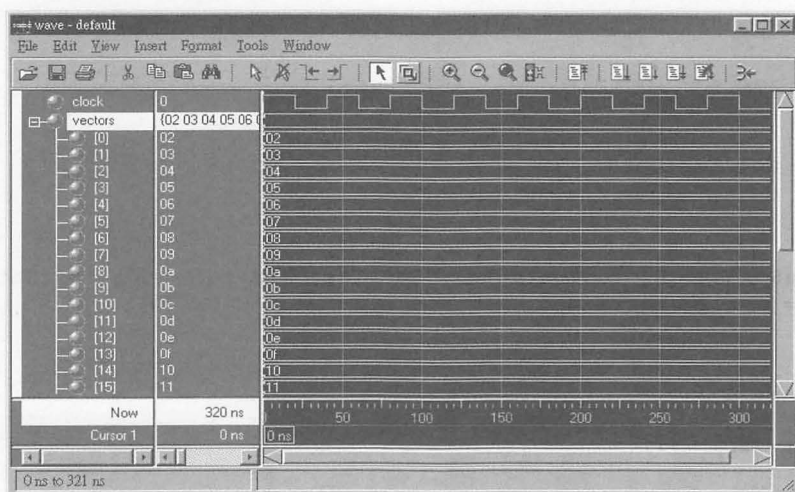
ModelSim SE 5.5 之後的版本就有支援 “Enhanced Invocation Option Test” 了。

新增加的系統函數(system function)：**\$value\$plusargs**。

Enhanced Invocation Option Test 的例子 1

```
verilog test.v chip.v +test2
或者是
vlog tb_In_Option_1.v
vsim tb_In_Option_1 +test2

INITIAL
begin
  if ($test$plusargs("test1")) // +test1
    $readmemh("test1.dat", vectors);
  else if ($test$plusargs("test2")) // +test2
    $readmemh("test2.dat", vectors);
  else if ($test$plusargs("test3")) // +test3
    $readmemh("test3.dat", vectors);
  else // 没有 +test1, +test2 或 +test3
    $display("Error: no test option specified");
end
```



Enhanced Invocation Option Test 的例子 2

```

verilog test.v chip.v +testfile=test2.dat
或者是
vlog tb_In_Option_2.v
vsim tb_In_Option_2 +testfile=test2.dat

reg [80:0] file_string;
reg [7:0] vectors[0:31];
initial
begin
  if ($value$plusargs("testfile=%s", file_string))
    $readmemh(file_string, vectors); // test2.dat
  else
    // 没有 +testfile=xxx.dat
    $display("Error: no test file option specified");
end

```

Enhanced Invocation Option Test 的例子 3

```

verilog test.v chip.v +TestData=2
或者是
vlog tb_In_Option_3.v
vsim tb_In_Option_3 +TestData=2

reg clock;
initial clock = 0;

```

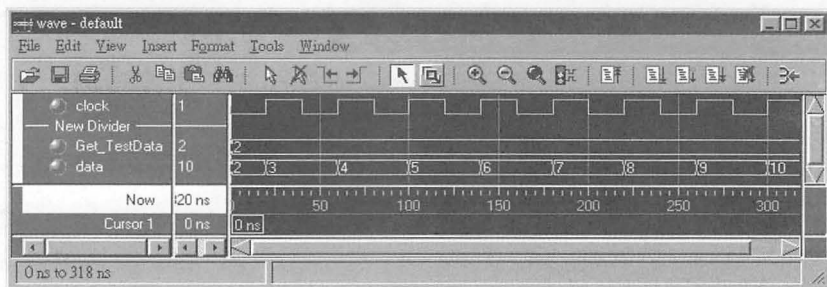
```

always
begin
    #20 clock = ~clock;
end

integer Get_TestData;
integer data;
initial
begin
    $value$plusargs("TestData=%d", Get_TestData); // Get_TestData=2
    data = Get_TestData;
end

always @(posedge clock)
begin
    data = data + 1;
end

```



13.28 On-detect Pulse Error Propagation

在 Verilog 1995 中，有“on-event pulse error propagation”的問題，當一個有效的輸入訊號改變的同時，如果有一個出現在 module 輸入的突波(glitch)，這個輸入的突波(glitch)也會用同樣的 delay 時間傳遞到輸出形成了‘X’。它的寬度比“路徑延遲”來得小。

在 Verilog 2001 中，增加了“on-detect pulse error propagation”，只要出現在 module 輸入的突波(glitch)被偵測到了，不需視“有效的輸入訊號改變的時間點”，‘X’就傳遞到輸出。

Verilog 2001 的 “On-detect Pulse Error Propagation” 是用於電路的 Model 設計，和是否能夠作為電路合成這個特性無關。

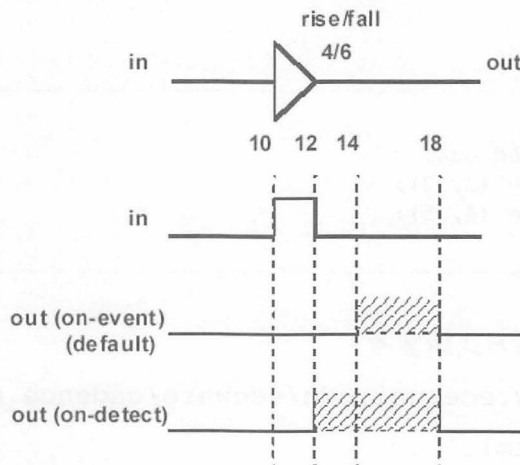
ModelSim SE 5.5 之後的版本就有支援 “On-detect Pulse Error Propagation” 了。

新增加的 Verilog 保留字：pulsestyle_onevent 以及 pulsestyle_ondetect。

簡單的例子：

```
specify
  pulsestyle_ondetect out;
  (in => out) = (4, 6);
endspecify
```

訊號 in 有 2 個單位的突波 (glitch)，on-event 需等 rise=4 個單位，而 on-detect 則是等 2 個單位 (和突波的寬度相同)。



其他詳細的使用方式請參考：

www.verilog-2001.com/verilog-2001_presentation.pdf



13.29 Negative Pulse Detection

在 Verilog 1995 中，出現在 module 輸入的 negative pulse 是會被拿掉不看的。

在 Verilog 2001 中，增加了偵測負向突波(negative pulse detection)，由於各個訊號由 0→1 的轉換延遲(rising-transition delay)或者是由 1→0 的轉換延遲(falling-transition delay)，有可能因而造成一個突波(glitch) 'x' 在 leading edge 之前被傳遞開來。

Verilog 2001 的“Negative Pulse Detection”是用於電路的 Model 設計，和是否能夠作為電路合成這個特性無關。

ModelSim SE 5.5 之後的版本就有支援“Negative Pulse Detection”了。

新增加的 Verilog 保留字：\$showcancelled 及 \$noshowcancelled。

簡單的例子：

```
specify
  showcancelled out;
  (a => out) = (2, 3);
  (b => out) = (4, 5);
endspecify
```

其他詳細的使用方式請參考：

http://www.ece.uci.edu/eceware/cadence_docs/vlogref/chap12.html



13.30 New Timing Constraint Check

Verilog 2001 中，增加新的“New Timing Constraint Check”，用來 model 非常深次微米電路的輸入系統規定參數(very deep submicron input constraints)。

Verilog 2001 的“New Timing Constraint Check”是用於電路的 Model 設計，和是否能夠作為電路合成這個特性無關。

SIMUCAD 的 Silos Verilog Logic Simulator，有支援“New Timing Constraint Check”。

新增加的 timing constraint 函數：**\$removal**、**\$recrem**、**\$timeskew** 以及 **\$fullskew**。

其他詳細的使用方式請參考：

www.verilog-2001.com/verilog-2001_presentation.pdf



13.31 Negative Timing Constraint

在 Verilog 2001 中，增加了可以給 cell 的 \$setphold(setup 以及 hold time) 以及 \$recrem(recovery 以及 removal time) “負數值”(negative value)。

對於原本 Verilog 1995 \$setphold 這個 task 多了用於給定 \$setphold “負數值”選擇性的參性。

Verilog 2001 的“Negative Timing Constraint”是用於電路的 Model 設計，和是否能夠作為電路合成這個特性無關。

SIMUCAD 的 Silos Verilog Logic Simulator，有支援“Negative Timing Constraint”。

新增加的 timing check 函數：**\$recrem**。

例如：

```
$recrem(posedge ctrl, clk, <recovery_limit>, <removal_limit>);
```

等於是

```
$removal(posedge ctrl, clk, <removal_limit>);
```

加上

```
$recovery(posedge ctrl, clk, <recovery_limit>);
```

其他詳細的使用方式請參考：

[http://www.ece.uci.edu/eceware/cadence_docs/vlogref/
chap13.html#1032243](http://www.ece.uci.edu/eceware/cadence_docs/vlogref/chap13.html#1032243)



13.32 Enhanced SDF support

在 Verilog 2001 標準中，定義了基於最新的 SDF 標準 (IEEE 1497-1999)，在 SDF 中的 timing object 如何對映到 Verilog 中的 cell、port... 等等的 object。

在 Verilog 2001 標準中，也改變了 specparam constant 的語法，讓 specparam constant 也能像 specify 區塊 (支援 SDF) 宣告在 module 層次中。

Verilog 2001 的 “Enhanced SDF support” 是用於電路的 Model 設計，和是否能夠作為電路合成這個特性無關。

新增加的系統函數(system function)：**\$sdf_annotate**。

ModelSim SE 5.5 之後的版本有支援 “Enhanced SDF support”。

sdf_annotate 的例子

```
$sdf_annotate("my design.sdf", testbench.u_design, , , "maximum");  
// 最後一個參數是 delay model, 可以是:  
// "minimum", "typical", "maximum", 或者是 "tool_control" 的其中一種
```

specparam 的例子

```
specify  
  // Pin-to-pin path delay  
  (in *> out) = 3;  
  
  // Timing constraint check  
  $setup(in, posedge clk, 1.2);  
  
  // specparam constant  
  specparam drive = 0.4; specparam load = 1.8;  
endspecify
```



13.33 Extended VCD File/PLI Enhancement

在 Verilog 2001 中，增加新的產生波型輸出(Value Change Dump，VCD)的能力，包括下列幾種：

- 輸出 port 值的更改(Dump port change value)
- 訊號強度(Dump strength level change)

- 電路模擬結束的時間(Dump the time at which simulation finishes)

新增加的系統函數(system function) : \$dumpports、\$dumpportsall、\$dumpportsoff、\$dumpportson、\$dumpportslimit 及 \$dumpportsflush。

其他詳細的使用方式請參考：

- (1) http://jason.sdsu.edu/modelsim/se_html/manual_html/se_man_XHTMLIX.html
- (2) 您的系統安裝 ModelSim SE 5.6x 之後，安裝在 C:\Modeltech_5.6d\docs\se_html\manual_html 裡頭的 c_vcnds2.html。
- (3) 您的系統安裝 ModelSim SE 5.7x 之後，安裝在 C:\Modeltech_5.7c\docs\se_html\SE Command Reference 裡頭的 c_vcnds2.html，要找其他的 SE Command，開啓 C:\Modeltech_5.7c\docs\se_html 裡頭的 wwhelp.htm 之後，用“索引”的方式去找比較方便。

Verilog 2001 的“Extended VCD File/PLI Enhancement”是不能合成的 VHDL 語法。

ModelSim SE 5.5 之後的版本就有支援“Extended VCD File/PLI Enhancement”了。

在 Verilog 2001 中，也對 Programming Language Interface (PLI) 的部份作了增強，包括下列幾種：

- VPI library 的增強
- Simulation control
- stop、finish、save、restart... 等等

- Support for new Verilog 2001 HDL constructs
Array of instances、attributes、signed arithmetic、recursive functions、enhanced file I/O...等等
- 更新 TF librariy 以及 ACC librariy ("PLI 1.0")
- 更正錯誤的部份
- 更正含糊不清的部份



13.34 Verilog 2001 新增的保留字、運算子、函數、compiler directive 以及 token

底下是 Verilog 2001 保留字的列表：

automatic	cell	config	design
endconfig	endgenerate	generate	genvar
instance	liblist	localparam	none
noshowcancelled	pulsetyle_ondetect	pulsetyle_onevent	showcancelled
use			

底下是 Verilog 2001 新增的運算子的列表：

>>>	<<<	**	
-----	-----	----	--

底下是 Verilog 2001 新增的資料型態轉換函數的列表：

\$signed()	\$unsigned()		
------------	--------------	--	--

底下是 Verilog 2001 新增加的**檔案處理**函數的列表：

\$ferror	\$fgetc	\$fgets	\$fflush
\$fread	\$fscanf	\$fseek	\$ftel
\$rewind	\$ungetc		

底下是 Verilog 2001 新增加的**字串處理**函數的列表：

\$sformat	\$swrite	\$swriteb	\$swriteh
\$swriteo			

底下是 Verilog 2001 新增的 **timing constraint** 函數的列表：

\$removal	\$recrem	\$timeskew	\$fullskew
-----------	----------	------------	------------

底下是 Verilog 2001 新增的 **timing check** 函數的列表：

\$recrem			
----------	--	--	--

底下是 Verilog 2001 新增加的**系統函數**(system function)的列表：

\$value\$plusargs	\$sdf_annotate	\$dumpports	\$dumpportsall
\$dumpportsoff	\$dumpportson	\$dumpportslimit	\$dumpportsflush

底下是 Verilog 2001 新增的 **compiler directive** 函數的列表：

`ifndef	`elsif	`line	`file
---------	--------	-------	-------

底下是 Verilog 2001 新增的 **token** 函數的列表：

(*	*)		
-----	----	--	--

本章練習

Question 1 :

有那些電路模擬或合成軟體支援 Verilog 2001 增強特色 ?

Question 2 :

什麼時候會用到 Verilog 2001 增強特色 ?

PS Google 可以幫您找到解答。

